

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

**MIDDLEWARE UNIFICADO SQL PARA BASES DE DATOS
NOSQL CLAVE-VALOR, GRAFOS Y GEOESPACIAL**

**DESARROLLO DE UN MIDDLEWARE PARA LA TRADUCCIÓN
DE CONSULTAS SQL A UN SISTEMA DE BASE DATOS
ORIENTADAS A GRAFOS**

**TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO
COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE
INGENIERO/A EN SOFTWARE**

EDWIN ANDRES CANTUÑA GUANO

edwin.cantuna@epn.edu.ec

DIRECTOR: VICTOR VICENTE VELEPUCHA BONETT

victor.velepucha@epn.edu.ec

DMQ, enero 2026

CERTIFICACIÓN

Yo, EDWIN ANDRES CANTUÑA GUANO declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

EDWIN ANDRES CANTUÑA GUANO

Certifico que el presente trabajo de integración curricular fue desarrollado por EDWIN ANDRES CANTUÑA GUANO, bajo mi supervisión.

PHD. VICTOR VICENTE VELEPUCHA BONETT
DIRECTOR

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

EDWIN ANDRES CANTUÑA GUANO

PHD. VICTOR VICENTE VELEPUCHA BONETT

SEBASTIAN ALEJANDRO SANCHEZ MALDONADO

JAIRO RENE SIMBAÑA CAIZA

DEDICATORIA

Dedico este trabajo a mis padres, Edwin Cantuña y Nancy Guano, quienes con su amor incondicional, su constante apoyo y esa incansable lucha que realizan día a día, guían cada paso que doy hacia mis metas y objetivos.

A mis hermanas, Stefany y Damaris, quienes con sus ocurrencias me dan motivos y fuerzas para seguir adelante en este camino y no rendirme.

A toda mi familia que con su apoyo y sus consejos han formado parte importante de este camino que está llegando a su fin.

Y en especial a Dios que estuvo junto a mí en cada momento, permitiéndome superar cada obstáculo que se iba presentando.

AGRADECIMIENTOS

Quiero expresar un enorme agradecimiento a todas las personas que han sido parte de esta etapa de mi vida, ya que sin ellas no habría sido posible culminarla.

En primer lugar a mis padres, Edwin y Nancy, nunca han dejado de luchar por mis hermanas y sobretodo por mí, me siento orgulloso del trabajo arduo que realizan por nuestro bienestar. Gracias por el cuidado y el amor incondicional que me brindan a diario. Estaré agradecido eternamente por todo lo que son conmigo.

A mis hermanas, Stefany, Damaris y a Omar, hemos compartido la mayor parte de esta vida entre risas, llantos, aciertos y errores que sé que nos ha hecho mejores personas. Su presencia y su bienestar son primordiales para mí.

También agradezco a toda mi familia con quienes he compartido muchos momentos de felicidad, en especial a mis tías, Gladys, Mónica, Geovana, Maribel y Fernanda, quienes han estado al pendiente de mí en todo momento, mis tíos, Fernando que me forjó en el trabajo arduo y Jorge que me incentivó en lo que hoy es mi sueño.

Hay 5 personas que fueron fundamentales en este proceso y agradezco mucho ese cariño y todas las veces que me consintieron desde que era niño: mi mamita María, papito Luis, abuelito Manuel, abuelita Agueda y mi angelito en el cielo que siempre me dijo que me quería ver graduado, mi abuelita Dioselina.

Hago una mención especial al club The Migers porque fue el equipo donde compartí con tíos, primos y grandes amigos muchas victorias y derrotas, pero sobretodo, grandes anécdotas que llevaré siempre conmigo.

A todos los amigos del colegio y de la universidad, con los que compartí grandes momentos y me supieron apoyar cuando más lo necesitaba.

A mi tutor de tesis, Víctor Velepucha, quien supo guiarme con mucha sabiduría y paciencia para realizar este proceso de la mejor manera.

Finalmente, agradezco al profesor Carlos Mena, por permitirme representar a la EPN durante todos estos años, y dejar el nombre de esta institución en alto, y a Emily que me inspiró con su bondad y su nobleza a ser una mejor persona.

Índice general

1. INTRODUCCIÓN	1
1.1. Objetivo general	2
1.2. Objetivos específicos	2
1.3. Alcance	2
1.4. Marco teórico	3
1.4.1. Antecedentes	3
1.4.2. Arquitectura de Software	4
1.4.3. Metodología de Desarrollo de Software	5
1.4.4. Frontend	7
1.4.5. Backend	10
1.4.6. Bases de Datos	11
1.4.7. Herramientas de desarrollo, gestión y pruebas	14
1.4.8. Pruebas de Software	15
2. METODOLOGÍA	17
2.1. Fase Exploratoria	17
2.1.1. Stakeholders	17
2.1.2. Roles de Scrum	18
2.1.3. Planificación del proyecto	18
2.2. Fase de Inicialización	19
2.2.1. Stack Tecnológico	19
2.2.2. Configuración del Proyecto	20
2.2.3. Sprint 0	29
2.3. Fase de Desarrollo	36
2.3.1. Sprint 1	36
2.3.2. Sprint 2	42

2.3.3. Sprint 3	45
2.3.4. Sprint 4	48
2.3.5. Sprint 5	52
2.3.6. Sprint 6	55
3. RESULTADOS, CONCLUSIONES Y RECOMENDACIONES	59
3.1. Resultados	59
3.1.1. Pruebas Funcionales	59
3.1.2. Pruebas de Usabilidad	60
3.1.3. System Usability Scalie (SUS)	62
3.1.4. Resultados de Pruebas Funcionales	64
3.2. Conclusiones	66
3.3. Recomendaciones	67
4. REFERENCIAS BIBLIOGRÁFICAS	68
Anexos	71

Índice de figuras

1.1. Arquitectura del middleware.	5
2.1. Planificación del proyecto. (reajustar.)	19
2.2. Diagrama de contexto del sistema.	31
2.3. Diagrama de contenedores del sistema.	31
2.4. Diagrama de base de datos propuesta.	32
2.5. Prototipo de pantalla de traducción realizado en Figma.	33
2.6. Configuración inicial del proyecto en Trello.	34
2.7. Distribución de repositorios en GitHub.	35
2.8. Implementación del Visitor para la traducción.	43
2.9. Componente de la página de traducción.	44
2.10. Nuevos métodos en el Visitor para DML.	47
2.11. Endpoint para solicitar restablecimiento de contraseña.	50
2.12. Página de gestión de conexiones.	53
2.13. Endpoint para obtener datos de analítica.	56
2.14. Componente del dashboard de administración.	57

Índice de Tablas

1.1. Roles de Scrum	6
1.2. Eventos de Scrum	6
1.3. Artefactos de Scrum	7
2.1. Partes interesadas del proyecto	18
2.2. Roles de Scrum	18
2.3. Stack tecnológico.	20
2.4. Épicas de Usuario del proyecto.	20
2.5. Historia de Usuario QTE-01	21
2.6. Valoración de prioridad.	22
2.7. Valoración de complejidad.	23
2.8. Product backlog del proyecto.	23
2.9. Planificación de Sprints.	28
2.10. Sprint 0 Backlog.	29
2.11. Sprint 1 Backlog.	36
2.12. Endpoints de autenticación y gestión de conexiones	40
2.13. Sprint 2 Backlog.	42
2.14. Sprint 3 Backlog.	46
2.15. Sprint 4 Backlog.	49
2.16. Sprint 5 Backlog.	52
2.17. Sprint 6 Backlog.	55
3.1. Caso de prueba: Traducir sentencia con operaciones INSERT	60
3.2. Tareas Específicas para medición de la Usabilidad del <i>middleware</i>	61
3.3. Preguntas SUS.	63
3.4. Escala de Likert.	63
3.5. Resultados de Tareas Específicas.	64

3.6. Resultados de la encuesta SUS.	65
---	----

RESUMEN

Este Trabajo de Integración Curricular (TIC) se centra en el desarrollo de un middleware traductor de sentencias SQL para sistemas de bases de datos orientadas a grafos. Actualmente, existe una brecha en la integración de aplicaciones tradicionales que manejan bases de datos SQL con tecnologías NoSQL, que hace que los desarrolladores de bases de datos tengan que aprender nuevos lenguajes de consulta, lo cual implica un tiempo adicional y reduce su productividad. Ante este problema, se propone como solución un sistema que traduzca sentencias SQL en su equivalente a un GQL como Cypher y la ejecución directa de esta traducción en una base de datos orientada a grafos sin la necesidad de ingresar a un GDBMS como Neo4j. Para una entrega de valor continua e incremental, se optó por el uso del marco de trabajo Scrum, estableciéndose 6 sprints para el desarrollo del middleware. El backend fue realizado con FastAPI incluye un parser SQL capaz de analizar la consulta ingresada, un traductor implementado mediante reglas de mapeo y los conectores respectivos tanto para la base de datos SQL como para la de Neo4j. Por su parte, el frontend fue construido con NextJS, que permitirá a los desarrolladores ingresar las sentencias SQL y visualizar tanto la traducción como los resultados de la ejecución de forma fácil e intuitiva. Con este trabajo se busca disminuir curva de aprendizaje hacia tecnologías desconocidas, y aumentar la interoperabilidad entre sistemas SQL y NoSQL contribuyendo a una migración más rápida y eficiente.

Palabras Claves - Parser SQL, Traductor SQL-NoSQL, Scrum, Neo4j, Cypher

ABSTRACT

This Curricular Integration Project (CIP) focuses on the development of middleware that translates SQL statements for graph-oriented database systems. Currently, there is a gap in the integration of traditional applications that handle SQL databases with NoSQL technologies, which means that database developers have to learn new query languages, which takes additional time and reduces their productivity. To address this problem, the proposed solution is a system that translates SQL statements into their equivalent in a GQL such as Cypher and executes this translation directly in a graph-oriented database without the need to enter a GDBMS such as Neo4j. For continuous and incremental value delivery, the Scrum framework was chosen, with six sprints established for the development of the middleware. The backend was built with FastAPI and includes an SQL parser capable of analyzing the query entered, a translator implemented using mapping rules, and the respective connectors for both the SQL database and the Neo4j database. The frontend was built with NextJS, which will allow developers to enter SQL statements and view both the translation and the execution results in an easy and intuitive way. This work seeks to reduce the learning curve for unfamiliar technologies and increase interoperability between SQL and NoSQL systems, contributing to faster and more efficient migration.

Keywords - SQL Parser, SQL-NoSQL Translator, Scrum, Neo4j, Cypher

Capítulo 1

INTRODUCCIÓN

El auge de los datos no estructurados y la necesidad de sistemas escalables horizontalmente han impulsado la creciente adopción de bases de datos NoSQL, las cuales ofrecen modelos de datos más flexibles y eficientes para grandes volúmenes de información en comparación con las bases de datos relacionales tradicionales. Sin embargo, la migración e integración de aplicaciones existentes, así como la curva de aprendizaje asociada a los diferentes lenguajes de consulta y APIs de cada base de datos NoSQL, representan un desafío significativo para los desarrolladores. Esta problemática dificulta una transición fluida desde entornos SQL bien establecidos hacia el paradigma NoSQL.

En este contexto, el componente desarrollado en este proyecto es un middleware SQL, diseñado para actuar como una capa de abstracción entre las aplicaciones que utilizan consultas SQL y los diversos sistemas de bases de datos NoSQL, específicamente aquellos de tipo orientados a grafos. La finalidad principal de este middleware es traducir las consultas SQL en operaciones compatibles con los lenguajes nativos de las bases de datos NoSQL seleccionadas, eliminando la necesidad de que los desarrolladores adquieran un conocimiento profundo en un lenguaje de consulta orientado a grafos como Cypher. Al proveer esta capa de traducción, el middleware simplifica la integración de aplicaciones existentes con entornos NoSQL y reduce significativamente la complejidad de la migración de datos y funcionalidades.

El desarrollo de este middleware implica la construcción de un parser SQL capaz de analizar y descomponer las consultas SQL en componentes manejables. Posteriormente, se implementará un conector para bases de datos orientadas a grafos utilizando el motor de Neo4j. Este conector será responsable de transformar la representación intermedia de las consultas SQL al lenguaje de consulta Cypher, propio de la base de datos orientada a

grafos Neo4j. Además, se desarrollará un conector de entrada para el middleware que recibirá las consultas SQL directamente, y una interfaz de usuario para que los desarrolladores puedan interactuar con el sistema, ingresar consultas SQL y visualizar los resultados traducidos y ejecutados en el sistema NoSQL apropiado. Este enfoque integral busca mejorar la interoperabilidad entre los paradigmas SQL y NoSQL, ofreciendo una solución práctica para gestionar la complejidad de los entornos de datos híbridos.

1.1. Objetivo general

Desarrollar un middleware que integre un parser SQL, un conector para base de datos orientada a grafos y un conector para bases de datos relacionales que permita traducir consultas SQL en operaciones compatibles con bases de datos orientadas a grafos a través de una interfaz de usuario que permita ingresar la consulta SQL e indique su equivalente en un lenguaje orientado a grafos.

1.2. Objetivos específicos

1. Desarrollar un parser SQL que analice y descomponga las consultas SQL en componentes manejables.
2. Implementar conectores para bases de datos orientadas a grafos, que traduzcan las consultas SQL a consultas GQL.
3. Desarrollar un conector que permita recibir consultas SQL y enviarlas al middleware. El middleware será responsable de traducir estas consultas SQL a una representación intermedia, facilitando su posterior conversión al lenguaje de consulta de grafos.

1.3. Alcance

El alcance del proyecto comprende el desarrollo de un middleware unificado SQL para bases de datos NoSQL, con capacidad para traducir consultas SQL en instrucciones compatibles con bases de datos orientadas a grafos, clave-valor y geoespaciales. Este componente incluye:

1. **Revisión de literatura y análisis de requisitos:** Se realizará una investigación exhaustiva sobre las técnicas y herramientas existentes para la traducción de consultas SQL

a NoSQL, así como un análisis de los requisitos necesarios para el desarrollo del middleware.

2. **Definición de la gramática SQL:** Utilizando la herramienta adecuada, se definirá y generará la gramática SQL necesaria para el análisis y descomposición de consultas SQL en componentes manejables.
3. **Desarrollo del parser SQL:** Implementación del parser SQL que analizará las consultas SQL y generará un árbol de análisis que servirá de base para la traducción a lenguajes de consulta NoSQL.
4. **Desarrollo de conectores NoSQL:** Implementación de un conector que traduzca consultas SQL al lenguaje de consulta Cypher. Este conector permitirá la ejecución de consultas traducidas, enfocándose en las relaciones y estructuras interconectadas.
5. **Desarrollo de conector:** Implementación de un conector que permita recibir consultas SQL y enviarlas al middleware. El middleware será responsable de traducir estas consultas SQL a una representación intermedia.
6. **Interfaz de usuario:** El middleware incluirá una interfaz de usuario que permitirá a los desarrolladores interactuar con el sistema de manera directa. A través de esta interfaz, los usuarios podrán ingresar consultas SQL y recibir los resultados traducidos y ejecutados en el sistema NoSQL correspondiente.

1.4. Marco teórico

1.4.1. Antecedentes

Dentro de los últimos años, el uso de bases de datos no relacionales en la implementación de sistemas de información ha crecido considerablemente motivo por el cual muchas empresas de desarrollo de software han optado su deseo de migrar las bases de datos SQL en sus soluciones a NoSQL, dado la flexibilidad, escalabilidad y variedad que brinda este tipo de bases de datos hacia sus intereses. Esta necesidad ha conllevado al desarrollo de soluciones que permitan realizar una eficiente traducción entre estos dos tipos de bases de datos. Para el presente trabajo, se ha realizado una búsqueda exhaustiva de anteriores trabajos referentes a la traducción de bases de datos relacionales a orientadas a grafos dentro de plataformas digitales como la Biblioteca Digital EPN, SCOPUS y Google Scholar.

Se utilizaron varias cadenas de búsqueda como “SQL a NOSQL”, “Traducción de SQL a NoSQL”, “Traducción SQL a grafos”, “Relacional a grafos” y “Relacional a NoSQL” con lo cual se encontraron algunos trabajos relevantes:

- Dai J [1], se centra en la transformación del esquema, la traducción y la optimización de consultas, especialmente en cargas de trabajo OLAP y bases de datos NoSQL orientadas a columnas. En este artículo indica que la desnormalización en NoSQL es clave para evitar costosas operaciones `JOIN` en entornos NoSQL y que técnicas como MapReduce son fundamentales en la traducción de consultas SQL a operaciones NoSQL.
- Namdeo y Suman [2], proponen un middleware para traducción de consultas SQL a NoSQL (SQL-No-QT) el cual actúa como capa intermedia entre aplicaciones heredadas y bases de datos MongoDB. Este modelo tiene la capacidad de traducir todas las consultas `CRUD` e incluso aquellas que contienen sentencias `JOIN`, de lo que carecía soluciones anteriores como UnityJDBC que se limitaban a traducir consultas `SELECT` sencillas.
- Dukic et al. [3], presenta un enfoque sistemático para poder convertir bases de datos relacionales a bases de datos de grafos, enfocándose especialmente en la migración a Neo4J. Esta metodología abarca tres fases: preparación, carga de datos y generación de relaciones, y optimización de la base de datos de grafos. A través de este enfoque se obtuvo una mayor eficiencia en el manejo de datos densamente relacionados así como tiempos de ejecución más cortos en comparación con las bases de datos relacionales.

1.4.2. Arquitectura de Software

La arquitectura de software es la forma de representar cómo se construye un sistema, dividiéndolo en componentes que se comunican entre sí. El propósito de diseñar una arquitectura eficaz es facilitar el entendimiento, desarrollo, despliegue y mantenimiento del producto por parte de los desarrolladores. De acuerdo con Martin [4], una buena arquitectura debe estar centrada en los casos de uso y ser plasmada sobre las funciones del sistema, en lugar de estar dominada por las tecnologías subyacentes.

Para el presente proyecto se definió una arquitectura modular para representar el middleware traductor de consultas SQL a su equivalente en Cypher, el lenguaje con el cual

trabaja las bases de datos de Neo4j. En la Figura 1.1, se puede visualizar la arquitectura adoptada para la realización de este proyecto. Aquí se identifican 4 componentes claves como son: Cliente, Frontend, Backend y las bases de datos, además se indica los protocolos de comunicación respectivos entre los componentes interrelacionados.

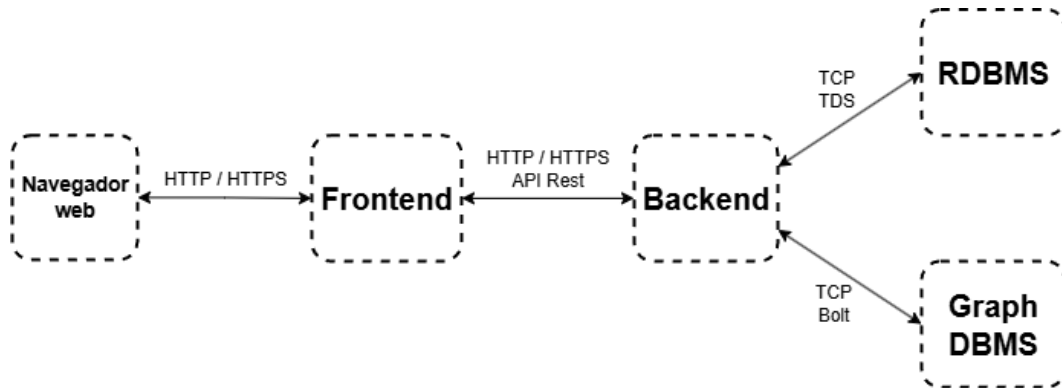


Figura 1.1: Arquitectura del middleware.

1.4.3. Metodología de Desarrollo de Software

Las metodologías de desarrollo de software son un conjunto de procesos, métodos y herramientas que permiten gestionar cada fase del ciclo de vida del desarrollo de un producto software. Su objetivo principal es organizar y estructurar el trabajo del equipo de desarrollo, garantizando que el proyecto se complete de manera eficiente [5]. A continuación se tratará acerca de dos enfoques muy conocidos como son las metodologías tradicionales y Agile.

Metodologías Tradicionales

De acuerdo con Pressman [5], este enfoque se caracteriza por seguir un proceso estricto y lineal, es decir, se requiere completar una fase para avanzar a la siguiente. Estos enfoques suelen aplicarse cuando los requerimientos se encuentran bien definidos y por tanto, los cambios serán mínimos, además de tener una documentación exhaustiva. Su única desventaja radica en la carencia de flexibilidad a los cambios.

Metodologías Ágiles

Por su parte, las metodologías ágiles se basan en un enfoque iterativo e incremental que prioriza la flexibilidad al cambio, el trabajo colaborativo y la entrega continua de valor. Se divide en ciclos cortos de trabajo con el propósito de que al finalizar cada ciclo se pueda

entregar una versión funcional del producto. Este enfoque fomenta la participación activa del cliente maximizando su satisfacción mediante la entrega rápida y constante de un producto de alta calidad [6].

Para cumplir de manera eficiente con los principios y valores de este tipo de metodologías, existen varios marcos de trabajo siendo Scrum el más conocido y el que será tratado a continuación.

Framework Scrum

Es un marco de trabajo que proporciona un conjunto de roles, eventos y artefactos para gestionar y organizar proyectos de desarrollo de software. Scrum permite una entrega de valor frecuente y una adaptación constante a las necesidades del negocio [7].

En la Tabla 1.1, Tabla 1.2 y Tabla 1.3, se resumen los roles, eventos y artefactos que proporcionan este marco de trabajo y permiten la construcción de un producto software funcional.

Tabla 1.1: Roles de Scrum

Rol	Descripción
Product Owner	Es el responsable de maximizar el valor del producto.
Scrum Master	Es el facilitador que guía al equipo en la correcta aplicación de Scrum.
Equipo de Desarrollo	Está encargado de entregar un incremento funcional del producto.

Tabla 1.2: Eventos de Scrum

Evento	Descripción	Duración
Sprint	Un ciclo de trabajo de duración fija, en el que se crea un incremento de producto potencialmente entregable.	Entre 2 y 4 semanas
Sprint Planning	La reunión donde el equipo planifica el trabajo a realizar durante el sprint y define su objetivo.	Hasta 2 horas por semana de Sprint

Continúa en la siguiente página...

Evento	Descripción	Duración
Daily Scrum	Una reunión diaria de 15 minutos para que el equipo sincronice sus actividades y adapte el plan del día.	15 minutos
Sprint Review	El encuentro donde el equipo inspecciona el incremento de producto con los stakeholders y obtiene retroalimentación.	Hasta 1 hora por semana de Sprint
Sprint Retrospective	Una reunión para que el equipo inspeccione su proceso de trabajo e identifique mejoras para el próximo sprint.	Hasta 45 minutos por semana de Sprint.

Tabla 1.3: Artefactos de Scrum

Artefacto	Descripción
Product Backlog	La lista priorizada de todo el trabajo necesario para el producto.
Sprint Backlog	El conjunto de elementos del Product Backlog seleccionados para el sprint.
Incremento	Los elementos completados en un sprint y los anteriores, listo para producción.

1.4.4. Frontend

El frontend de una aplicación web es la parte con la que los usuarios interactúan directamente. Su propósito principal es presentar la interfaz de usuario (UI) y añadir interactividad. Las tres lenguajes fundamentales e indispensables para el desarrollo frontend son HTML (Hypertext Markup Language), que define la estructura y la semántica de una página web; CSS (Cascading Style Sheets), que se encarga del diseño y la presentación; y JavaScript, que hace que las páginas web sean interactivas y dinámicas, permitiendo funciones como la clasificación de datos o la validación de formularios en el lado del cliente. En aplicaciones web modernas, como las aplicaciones de una sola página (SPA) implementadas con librerías o frameworks como React, Angular o Vue, gran parte de la lógica de presentación se ejecuta en el cliente, lo que las convierte en "clientes gruesos". Además, el frontend utiliza

APIs web, como el DOM API, para manipular elementos HTML, y el Fetch API o Ajax para cargar datos de manera asíncrona desde el servidor, optimizando la experiencia del usuario [8].

TypeScript

Es un lenguaje de programación que contiene la misma sintaxis base de JavaScript razón por la que es conocido como el “superset de JavaScript”. Es un sistema caracterizado por cuatro elementos clave [9]:

- Extiende la sintaxis de JavaScript con características específicas para definir y usar tipos.
- Tiene un verificador de tipos que examina el código para detectar configuraciones incorrectas de variables y funciones.
- Su compilador ejecuta el verificador de tipos, reporta cualquier problema y luego genera el código JavaScript equivalente.
- Dispone de un servicio de lenguaje que ofrece herramientas útiles a los desarrolladores, garantizando la legibilidad y mantenibilidad del código.

Para Goldberg [9], la adopción de TypeScript aporta beneficios sustanciales al desarrollo web al mitigar las debilidades de JavaScript, como la falta de seguridad y la documentación imprecisa. Ofrece seguridad al permitirnos especificar explícitamente los tipos de valores para parámetros y variables, lo que ayuda a prevenir errores en tiempo de ejecución que de otro modo podrían causar fallos en JavaScript. Esta restricción controlada sobre el código garantiza que los cambios en una sección no afecten inesperadamente a otras. Además, TypeScript facilita una documentación precisa al integrar la sintaxis para describir la forma de los objetos, creando un sistema de tipado robusto y obligatorio. Las herramientas de desarrollo (developer tooling) se ven significativamente mejoradas, ya que la información de tipos permite a los editores proporcionar sugerencias inteligentes de autocompletado y facilitar refactorizaciones complejas. A pesar de ser un lenguaje en constante evolución, el código TypeScript se transpila a JavaScript y todas las anotaciones y alias de tipos se eliminan en el proceso, sin añadir sobrecarga al tiempo de ejecución.

React

Es una biblioteca de JavaScript declarativa que facilita construcción de interfaces de usuario. Se centra en la composición de componentes, el manejo del estado y un flujo de datos unidireccional. La arquitectura de React fomenta una jerarquía de componentes, donde las partes más grandes de la interfaz de usuario se dividen en subcomponentes más pequeños, siguiendo principios como el de responsabilidad única. Esta modularidad facilita la construcción de interfaces complejas y su mantenimiento [10]. La comunicación entre componentes en React se gestiona principalmente a través de dos conceptos:

- **Props:** Son la manera de enviar datos de un componente padre hacia sus componentes hijos. Se pasan a una función como argumentos. El uso de props hace que un componente padre pueda personalizar la apariencia y el comportamiento de sus componentes hijos, y una vez que se pasan, no pueden ser modificados por los componentes hijos.
- **State:** El estado representa el conjunto mínimo de datos cambiantes que su aplicación necesita recordar para que la interfaz de usuario sea interactiva. El estado permite que un componente realice un seguimiento de la información y la cambie en respuesta a las interacciones del usuario. El principio más importante para estructurar el estado es mantenerlo DRY, identificando la representación mínima absoluta del estado y calculando todo lo demás bajo demanda. Para determinar si algo debe ser estado, se considera si cambia con el tiempo, si se pasa como prop desde un padre o si se puede calcular a partir del estado o props existentes.

NextJS

NextJS es un framework robusto utilizado para la creación de aplicaciones web mediante el uso de componentes React y otras funciones adicionales, logrando que estas sean dinámicas e interactivas. Entre sus principales características están el soporte de renderizado del lado del servidor, la generación de sitios estáticos y la fomentación de buenas prácticas dentro del desarrollo web, como rendimiento, SEO y seguridad [11].

Tailwind CSS

Es un framework CSS que proporciona una vasta colección de clases de utilidad de bajo nivel. Estas clases tienen un propósito único el cual es construir interfaces de usuario completas sin necesidad de escribir directamente reglas CSS en archivos separados. Simplemente se aplican las clases de utilidad a los elementos HTML a través de su atributo class. Además, facilita la implementación de diseños responsivos mediante prefijos de puntos de interrupción y permite la reutilización de estilos y la adhesión a la estrategia DRY [12].

Este framework soporta un alto nivel de personalización y permite manejar eventos y estados directamente a través de clases de utilidad.

1.4.5. Backend

El backend es la parte de una aplicación web que se ejecuta en el servidor y maneja la lógica de la aplicación y la gestión de datos. A diferencia de las páginas estáticas que solo sirven archivos, las páginas web dinámicas requieren que el servidor genere contenido en tiempo real, a menudo recuperando información de una base de datos. Las tecnologías de backend incluyen lenguajes de programación como JavaScript, PHP, Java, Python o Ruby, servidores web como nginx o Apache HTTP Server, y bases de datos, que pueden ser relacionales o NoSQL [8].

Python

Es uno de los lenguajes de programación más populares en la actualidad, bastante utilizado en distintas áreas como en el desarrollo web, la ciencia de datos y el Machine Learning dada su eficiencia, la interoperabilidad y facilidad para aprender [13]. En Python se puede utilizar bibliotecas predefinidas para el trabajo con bases de datos y la validación de la información contenida. A continuación se presenta algunas bibliotecas útiles para el desarrollo del middleware:

- **Biblioteca Pydantic:** Esta biblioteca de Python permite validar datos mediante type hints para lo cual, se definen modelos para verificar y convertir datos automáticamente. Con esto, mejora la calidad de los datos y la robustez de aplicaciones Python.
- **Biblioteca Pyodbc:** Esta biblioteca ayuda a la conexión de aplicaciones Python con

bases de datos vía ODBC. Además, permite ejecutar consultas de forma eficiente desde distintos motores de bases de datos como SQL Server, PostgreSQL, MySQL entre otros.

- **Biblioteca sqlparse:** Es una biblioteca de Python utilizado para analizar, validar y generar consultas SQL.

Para el desarrollo de aplicaciones web existen frameworks conocidos que fueron creados en Python tales como Django, Flask o FastApi.

FastAPI

FastAPI es un framework web moderno y de alto rendimiento que se distingue por un conjunto de características que lo hacen altamente eficiente para el desarrollo de APIs [14]:

- Es uno de los frameworks web más rápidos en Python dado su rendimiento y por soportar la programación asíncrona permitiendo el manejo de grandes volúmenes de solicitudes con latencia mínima.
- Utiliza Python type hints y modelos Pydantic para la validación automática de datos, garantizando que los datos de entrada se validen con precisión según los tipos y restricciones especificados, lo que lleva a un código más seguro.
- FastAPI genera automáticamente documentación API interactiva a través del uso de interfaces como Swagger UI y ReDoc. Con esto, permite la prueba y exploración de endpoints en tiempo real desde el navegador, facilitando la comprensión de la API y aumentando la mantenibilidad del código.
- Utiliza el servidor ASGI Uvicorn, el cual se distingue por su ligereza y su alto rendimiento en aplicaciones web asíncronas.
- Reduce los errores y aumenta la velocidad de desarrollo a través de funciones de autocompletado y verificación de tipos en el editor.

1.4.6. Bases de Datos

De acuerdo con Garcia-Molina [15], una base de datos es una colección organizada de información que puede persistir durante mucho tiempo y es administrada por un Sistema

Gestor de Bases de Datos (DBMS, por sus siglas en inglés). Un DBMS tiene como objetivo almacenar, recuperar y gestionar esta información de manera eficiente [16].

Bases de Datos Relacionales

Las bases de datos relacionales conforman el tipo más común de almacenamiento de información que ha existido hasta la fecha. Son un conjunto de tablas con datos que comparten atributos similares y pueden establecer relaciones con otras tablas mediante algún atributo [17]. Para garantizar confiabilidad en las transacciones, estas bases de datos cumplen con principios ACID:

- **Atomicidad:** Una transacción es “todo o nada”; o se completa por completo, o no se realiza ninguna de sus partes.
- **Consistencia:** Cada transacción debe llevar la base de datos de un estado válido a otro estado válido.
- **Aislamiento:** Las transacciones concurrentes no deben interferir entre sí; para el usuario, parece que se ejecutan una tras otra.
- **Durabilidad:** Una vez que una transacción se confirma, sus cambios son permanentes y persisten incluso si hay fallas en el sistema.

SQL Server

SQL Server es un sistema de gestión de bases de datos relacionales desarrollado por Microsoft. Utiliza SQL como su lenguaje principal y soporta diversos tipos de datos incluyendo numéricos, caracteres, entre otros [17]. Para la administración de infraestructuras SQL como SQL Server y Azure SQL Database SQL, Microsoft dispone del entorno SQL Server Management Studio (SSMS). Esta herramienta permite acceder, configurar, administrar y desarrollar todos los componentes de servidores SQL además de poder administrar bases de datos, realizar consultas y auditorías mediante scripts.

Bases de Datos NoSQL

Las bases de datos no relacionales, también conocidas como NoSQL (Not only SQL) ofrecen una alternativa de almacenamiento y gestión de grandes volúmenes de datos ya

que no requiere una estructura formal para agregarlos [18]. Con esto proporciona mayor flexibilidad al tratar con datos no uniformes o campos personalizados [19].

Existen diferentes tipos de bases de datos NoSQL entre los que se encuentran [20]:

- **Clave-valor:** Se almacenan como pares, siendo la clave el identificador único del valor asociado.
- **Documentos:** Almacenan datos en documentos semiestructurados especialmente en formatos como JSON o XML.
- **Familias de Columnas:** Organizan los datos en filas que tienen muchas columnas asociadas con una clave de fila.

Bases de Datos Orientadas a Grafos

Las bases de datos orientadas a grafos representan una categoría importante de bases no relacionales al modelar los datos como una red de nodos y relaciones [21]. Este enfoque brinda alternativas para escenarios donde la complejidad de los datos reside en sus conexiones y no solo en el volumen de estos. Este modelo abarca cuatro componentes fundamentales [22]:

- **Nodos:** Almacenan información de las entidades.
- **Relaciones:** Conectan a los nodos de forma explícita, es decir, tiene un tipo, un nodo origen, un nodo destino y una dirección.
- **Propiedades:** Son pares clave-valor que permite agregar atributos tanto a nodos como a relaciones.
- **Etiquetas:** Permiten agrupar y categorizar nodos.

Neo4j

Es una base de datos de grafos diseñada para gestionar y recorrer grandes cantidades de datos conectados con facilidad. Su infraestructura y formato de almacenamiento optimizado para el manejo de grafos ofrece mejoras significativas de velocidad y rendimiento con respecto a otras bases de datos [23]. Tiene un lenguaje de consulta propio llamado Cypher. Este permite a los usuarios describir el patrón de datos que desean encontrar facilitando las optimizaciones de búsqueda y mejorando la legibilidad [24].

Neo4j Desktop y Neo4j Browser

Estas herramientas sirven para gestionar bases de datos Neo4j. Neo4j Desktop permite la exploración y uso de este tipo de bases de datos de forma local. Por otro lado, en Neo4j Browser se puede visualizar los resultados de consultas sobre una base de datos específica a través de scripts y manipular los datos que contiene, ya sea los nodos o relaciones [24].

1.4.7. Herramientas de desarrollo, gestión y pruebas

Figma

Es una herramienta de diseño colaborativa basada en el navegador, que permite a los usuarios trabajar juntos en tiempo real en la misma interfaz de usuario. Destaca por sus potentes componentes reutilizables y su sistema flexible de anulaciones. Ofrece entrega integrada para desarrolladores, permitiendo copiar fragmentos de código y activos directamente del diseño. Además, facilita el prototipado, ya que los diseños siempre están en la nube, eliminando la necesidad de cargas manuales [25].

Git

Git es un sistema de control de versiones distribuido que almacena datos como una serie de instantáneas del proyecto. Su modelo de ramificación es una característica fundamental, permitiendo crear y destruir ramas de forma rápida y sencilla para aislar el trabajo en temas específicos. Permite trabajar con múltiples repositorios remotos y operar sin conexión. Git se utiliza principalmente a través de la línea de comandos para acceder a toda su funcionalidad, ofreciendo además herramientas para reescribir la historia localmente antes de compartirla [26].

GitHub

GitHub es el mayor proveedor de alojamiento de repositorios Git y un punto central para la colaboración de desarrolladores en proyectos de código abierto. Su flujo de trabajo se centra en las Pull Requests (PRs), que facilitan la discusión, revisión y fusión de cambios de manera colaborativa directamente en la plataforma web. Permite a los usuarios bifurcar proyectos, trabajar en sus propias copias y luego solicitar la integración de sus cambios.

GitHub soporta Markdown con características especiales como listas de tareas y fragmentos de código, además de ofrecer un sistema de notificaciones y funcionalidades como la automatización de tareas [26].

Visual Studio Code

Visual Studio Code es un editor de código potente y completo que funciona en Mac, Windows y Linux. Una de sus principales ventajas es su comprensión de todo el proyecto, incluyendo la relación entre archivos y soporte para múltiples lenguajes como Python, JavaScript y CSS. Ofrece autocompletado de código, navegación y refactorización inteligentes, y facilita la gestión de entornos virtuales y dependencias. Visual Studio Code también integra control de versiones para sistemas como Git, y herramientas para el desarrollo web frontend y backend, razón por la cual ha sido la elegida para el desarrollo de este trabajo [27].

Postman

Esta herramienta diseñada para la prueba y el desarrollo de APIs, permite a los usuarios crear, organizar y enviar solicitudes API en colecciones. Ofrece amplias opciones de autorización para APIs, incluyendo Basic Auth, Bearer Tokens, y OAuth, lo que facilita la interacción con diversas seguridades. Permite la creación de scripts de validación de pruebas y soporta pruebas basadas en datos para escalar los tests. Además, Postman facilita el diseño de especificaciones API, como OpenAPI, y puede generar automáticamente mocks y pruebas a partir de estas especificaciones. La herramienta también soporta la documentación de APIs directamente en la aplicación y la gestión de variables [28].

1.4.8. Pruebas de Software

Son una actividad fundamental en la ingeniería de software, cuyo objetivo es demostrar que un programa hace lo que se pretende y descubrir defectos antes de que el sistema se use. Al probar el software, se ejecuta el programa con datos artificiales y se verifican los resultados para buscar errores, anomalías o información sobre los atributos no funcionales del programa. Es importante destacar que las pruebas solo pueden mostrar la presencia de errores, no su ausencia. Siempre existe la posibilidad de que una prueba pasada por alto revele más problemas con el sistema [29].

Pruebas Funcionales

Las pruebas funcionales se refieren directamente a la verificación de los servicios y funcionalidades específicas que el sistema debe proveer. Se basan en los requerimientos funcionales, que son enunciados que describen cómo el sistema debe reaccionar a entradas particulares y cómo debe comportarse en situaciones específicas. Pueden ser descripciones abstractas o muy detalladas de las funciones del sistema, sus entradas, salidas y excepciones. El objetivo de estas pruebas es demostrar que el software ha implementado adecuadamente sus requerimientos [29].

Pruebas No Funcionales

Las pruebas no funcionales se ocupan de las limitaciones o restricciones sobre los servicios o funciones que ofrece el sistema. Estas restricciones pueden relacionarse con propiedades emergentes del sistema como la fiabilidad, el rendimiento, la seguridad, la usabilidad y la protección. Son a menudo más críticas que los requerimientos funcionales individuales, ya que el fracaso en cumplir con un requerimiento no funcional puede hacer que todo el sistema sea inútil. Siempre que sea posible, los requerimientos no funcionales deben escribirse de manera cuantitativa y medible, para que puedan ser probados objetivamente [29].

Capítulo 2

METODOLOGÍA

Este capítulo describe el desarrollo del *middleware* de traducción de consultas SQL a un sistema de base de datos orientada a grafos. Para una entrega de valor rápida y continua, el sistema será construido utilizando el marco de trabajo Scrum, el cual se caracteriza por la adaptabilidad y flexibilidad que brinda ante los cambios que se presenten a lo largo del proceso. Los roles, ceremonias y artefactos que forman parte de Scrum permitirán una organización y comunicación eficiente entre los integrantes del equipo de trabajo.

Este capítulo está dividido en tres secciones, donde se detallan las actividades y artefactos realizados para asegurar que la construcción del producto software se encuentre alineado a los objetivos de negocio.

2.1. Fase Exploratoria

Esta sección cubre aspectos relacionados a la identificación de todos los involucrados en el proyecto, es decir, tanto los stakeholders como los miembros del equipo Scrum y la planificación para la implementación del componente.

2.1.1. Stakeholders

Como punto de partida de la fase exploratoria, se identifican las partes interesadas (stakeholders) del proyecto, quienes cumplen un papel importante al ayudar en la comprensión de las funcionalidades que el *middleware* debe cubrir, esenciales para el adecuado diseño de la arquitectura, interfaces de usuario y la codificación. La Tabla 2.1 describe la responsabilidad que corresponde a cada stakeholder:

Tabla 2.1: Partes interesadas del proyecto

ID	Stakeholder	Descripción
S1	Usuario Final	Persona que interactuará directamente con el sistema, al ingresar sentencias SQL y visualizar la traducción correspondiente con los resultados de ejecución obtenidos.
S2	Docente	Persona encargada de supervisar y guiar en la construcción del producto software de acuerdo con el alcance acordado.
S3	Desarrollador	Será el responsable del diseño y el desarrollo de la aplicación asegurando el cumplimiento de los requisitos determinados.

2.1.2. Roles de Scrum

De acuerdo con Scrum, uno de los componentes indispensables es el equipo Scrum, dentro del cual existen roles que cada individuo cumplirá. En las primeras reuniones realizadas con el director y el resto de integrantes de este Trabajo de Integración Curricular (TIC), se asignaron los roles, tal y como se observa en la Tabla 2.2.

Tabla 2.2: Roles de Scrum

Rol	Persona(s) Asignada(s)
Product Owner	Víctor Velepucha
Scrum Master	Víctor Velepucha
Equipo de Desarrollo	Andrés Cantuña, Sebastián Sánchez y René Simbaña

2.1.3. Planificación del proyecto

Teniendo claro la visión del producto, los stakeholders y los roles de Scrum de los integrantes del equipo, se continuó con la planificación inicial del proyecto, mismo en el que se contemplaron 358 (no oficial) horas de trabajo por un periodo de 5 meses. Las actividades definidas incluyen el análisis de requerimientos, diseño, implementación y pruebas que se realizarán al sistema, además de la documentación y las ceremonias Scrum.

En la Figura 2.1 se visualiza el número de horas que llevó completar cada actividad.

	ACTIVIDADES ESPECÍFICAS	MES	MES 1				MES 2				MES 3				MES 4				MES 5				HORAS
		SEMANA	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	
1	Revisión sistemática de la literatura sobre gramáticas, parsers y técnicas de traducción SQL a grafos.		20																				20
2	Elaboración de las historias de usuario.			10																			10
3	Diseño de la arquitectura del middleware.			10																			10
4	Diseño de la interfaz de usuario.				8																		8
5	Definición de la gramática SQL.					10	10																20
6	Desarrollo del parser SQL.					10	10	10	10	10													50
7	Diseño del conector para bases de datos SQL.								10	10													20
8	Diseño del conector para bases de datos orientadas a grafos.										20	20	10										50
9	Implementación de conectores para traducir consultas SQL a lenguaje de consulta de grafos.													20	20	10							50
10	Desarrollo de la interfaz de usuario.									10							10						20
11	Pruebas de integración y funcionales al sistema.																	10	10				20
12	Pruebas no funcionales al sistema.																		10				10
13	Documentación del sistema.																			5	5	5	15
	Daily Scrum		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1				17
	Sprint Planning		4			4			4			4			4			2					22
	Sprint Review						2			2			2			2		2					10
	Sprint Retrospective				1		1			1			1			1		1					6
TOTAL HORAS																							358

Figura 2.1: Planificación del proyecto. (reajustar.)

Para asegurar que el desarrollo del *middleware* sea progresivo, se llevaron a cabo reuniones semanales con el Ph.D. Víctor Velepucha, quien también brindó una basta retroalimentación y seguimiento para que el proyecto mantenga una orientación alineada a los objetivos establecidos.

2.2. Fase de Inicialización

Esta sección abarca la selección del stack tecnológico, redacción de las Historias de Usuario (HU) que describan las funcionalidades de la aplicación, elaboración del Product Backlog, planificación de sprints y configuración inicial del entorno de trabajo, asentando una base sólida para la gestión y construcción eficiente del *middleware*.

2.2.1. Stack Tecnológico

La elección de tecnologías se hizo en base a la experiencia adquirida en el desarrollo de proyectos anteriores, la versatilidad y facilidad que brindan para ser integrados entre sí, útil para construir un producto altamente mantenible y escalable. La Tabla 2.3 indica las herramientas elegidas para cumplir con los eventos y artefactos correspondientes a Scrum.

Tabla 2.3: Stack tecnológico.

Herramienta	Descripción
Figma	Utilizado para el prototipado de interfaces de usuario del sistema.
Visual Studio Code	Editor de código utilizado para el desarrollo del sistema debido a su versatilidad y compatibilidad con múltiples lenguajes y frameworks.
Postman	Herramienta que permitirá evaluar los endpoints del <i>middleware</i> para asegurar su correcto funcionamiento e integración con el frontend.
Git y GitHub	Serán utilizados para el control de versiones y almacenamiento del código del proyecto.

2.2.2. Configuración del Proyecto

Épicas e Historias de Usuario

Luego de un análisis riguroso de los requisitos establecidos para desarrollar el proyecto TIC, fueron traducidos a Épicas de Usuario que representan las principales funcionalidades de la aplicación. La Tabla 2.4 describe las épicas definidas.

Tabla 2.4: Épicas de Usuario del proyecto.

ID	Épica	Descripción	HUs
AUM	Autenticación y gestión de usuarios	Facilita a todo desarrollador registrarse e iniciar sesión. Además brinda a los administradores, el control sobre los usuarios registrados para otorgar o denegar permisos dentro del sistema.	AUM-01, AUM-02, AUM-03 y AUM-04
CON	Gestión de conexiones	Permite al usuario configurar las conexiones a los DBMS SQL Server y Neo4j para la interacción con las bases de datos correspondientes.	CON-01 y CON-02

Continúa en la siguiente página...

ID	Épica	Descripción	HUs
QTE	Traducción y ejecución de consultas	Se encarga del análisis, traducción de sentencias SQL a lenguaje Cypher y su posterior ejecución en una base de datos orientada a grafos. También incluye el historial de consultas y la visualización de resultados obtenidos en distintos formatos.	QTE-01, QTE-02, QTE-03 y QTE-04
OBS	Observabilidad y seguimiento	Permite a los administradores la realización de auditoría y monitoreo de rendimiento para identificar áreas de mejora.	OBS-01 y OBS-02

Cada épica fue desglosada en Historias de Usuario con funcionalidades específicas. En la tabla 2.5 se presenta el formato elegido para todas las historias de usuario que se encuentran dentro del Anexo I. Entre los campos principales destacan la prioridad, riesgo, estimación, historia de usuario como tal o los criterios de aceptación.

Para obtener la estimación se utilizó la técnica **Planning Poker**, misma que permite a cada miembro del equipo de desarrollo elegir una puntuación basada en la escala de Fibonacci. La puntuación final es determinada mediante consenso, una vez que todos los individuos expongan las razones de su votación.

Tabla 2.5: Historia de Usuario QTE-01

ID: QTE-01	Usuario: Desarrollador
Título: Traducción de operaciones DML básicas	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alto
Estimación: 8	Iteración asignada: Sprint 1
Historia de Usuario: Como desarrollador, Quiero traducir sentencias SELECT a lenguaje Cypher, Para obtener datos específicos de una base de datos orientada a grafos.	

Continúa en la siguiente página...

Criterios de aceptación:

- El sistema deberá traducir la sentencia `SELECT` para todas las columnas o solo columnas específicas de una tabla.
- El sistema deberá soportar la cláusula `WHERE`, operadores de comparación (`=`, `<`, `<=`, `>`, `>=`, `<>`) y operadores lógicos (`AND`, `OR` y `NOT`).
- El sistema deberá traducir la cláusula `ORDER BY` junto a la forma de ordenamiento (`ASC` o `DESC`).
- El sistema deberá traducir correctamente las palabras reservadas `DISTINCT`, y `LIMIT/TOP`.
- El usuario deberá ingresar una sentencia SQL que cumpla con las restricciones indicadas.
- El usuario deberá seleccionar la opción **Traducir** para visualizar la sentencia en lenguaje Cypher equivalente a la que ingresó.
- Si la sentencia tiene errores de sintaxis o no es soportada, el sistema indicará un mensaje de error y no podrá ser traducida.
- Las consultas traducidas, serán agregadas automáticamente al historial.

Product Backlog

Para poder cumplir con un trabajo organizado y eficaz cada HU fue dividida en tareas específicas a través de un análisis riguroso y colectivo entre todo el equipo Scrum. Cada tarea fue valorada en términos de prioridad y complejidad utilizando las escalas de valoración que se encuentran en las Tablas 2.6 y 2.7.

La prioridad es el impacto que cada tarea tiene para satisfacer las necesidades del negocio. En este caso el rango fue dado de 1 a 3, siendo 1 la de mayor relevancia y 3 la de menor impacto.

Tabla 2.6: Valoración de prioridad.

Valor	Prioridad
1	Alta
2	Media
3	Baja

Por su parte, la complejidad radica en la complejidad que conlleva la ejecución de las tareas, de acuerdo a los recursos utilizados, el tiempo y esfuerzo empleado. El rango fue definido utilizando una vez más la serie de Fibonacci, donde 1 indica una tarea con muy poca complejidad y 8 para las tareas de mayor esfuerzo.

Tabla 2.7: Valoración de complejidad.

Valor	Complejidad
1	Muy baja
2	Baja
3	Media
5	Alta
8	Muy alta

La Tabla 2.8 expone el Product Backlog generado y valorado bajo los criterios mencionados. Esto favorecerá en la asignación de tareas durante la planificación de sprints.

Tabla 2.8: Product backlog del proyecto.

Cód. HU	Cód. Tarea	Tarea	Prioridad	Complejidad
AUM-01	T01	Implementar funcionalidad para registro de usuario.	1	2
	T02	Implementar funcionalidad para inicio de sesión.	1	5
	T03	Implementar validación de campos y credenciales para autenticación.	1	3
	T04	Implementar endpoints para registro e inicio de sesión.	1	3
	T05	Diseñar pantalla de registro de usuario.	2	2
	T06	Desarrollar pantalla de registro de usuario.	1	3
	T07	Diseñar pantalla de inicio de sesión.	2	1
	T08	Desarrollar pantalla de inicio de sesión.	1	2

Continúa en la siguiente página...

Cód. HU	Cód. Tarea	Tarea	Prioridad	Complejidad
AUM-02	T09	Implementar funcionalidad para actualización de perfil.	1	3
	T10	Implementar validación de campos modificados.	1	1
	T11	Implementar un endpoint para actualizar información.	1	2
	T12	Diseñar pantalla de actualización de perfil.	2	1
	T13	Desarrollar pantalla de actualización de datos.	1	2
AUM-03	T14	Implementar funcionalidad para restablecimiento de contraseña.	1	3
	T15	Implementar validación de email asociado a una cuenta.	1	2
	T16	Implementar validación de formato para nueva contraseña.	1	2
	T17	Diseñar formularios para solicitar correo y restablecer contraseña.	2	1
	T18	Desarrollar formularios para solicitud de correo y restablecimiento de contraseña.	1	3
AUM-04	T19	Implementar funcionalidad para gestión de usuarios.	1	5
	T20	Implementar endpoints correspondientes para administración de usuarios.	1	8
	T21	Diseñar pantalla de gestión de usuarios.	1	3
	T22	Desarrollar pantalla de gestión de usuarios.	1	5

Continúa en la siguiente página...

Cód. HU	Cód. Tarea	Tarea	Prioridad	Complejidad
CON-01	T23	Implementar funcionalidad para configurar las conexiones a las bases de datos.	1	3
	T24	Implementar pruebas de conectividad y funciones de conexión / desconexión.	1	8
	T25	Implementar un endpoint para el almacenamiento de una conexión.	1	3
	T26	Diseñar modal para configurar una conexión.	2	2
	T27	Desarrollar modal para configurar una conexión.	1	3
CON-02	T28	Implementar funcionalidad para administración de conexiones.	1	5
	T29	Implementar endpoints para la gestión de conexiones.	1	5
	T30	Diseñar pantalla de gestión de conexiones.	2	2
	T31	Desarrollar pantalla para administrar conexiones.	1	3
QTE-01	T32	Definir reglas léxicas y sintácticas de la sentencia SELECT, las cláusulas WHERE, ORDER BY, palabras reservadas como FROM, DISTINCT, LIMIT, operadores lógicos y de comparación.	1	8
	T33	Implementar funcionalidad para traducción de consultas SELECT.	1	8
	T34	Implementar un endpoint para la traducción de consultas.	1	3
	T35	Diseñar pantalla de traducción.	1	3

Continúa en la siguiente página...

Cód. HU	Cód. Tarea	Tarea	Prioridad	Complejidad
	T36	Desarrollar pantalla de traducción de consultas.	1	5
QTE-02	T37	Extender gramática para funciones de agregación como COUNT, SUM, MIN, MAX o AVG, cláusulas ORDER BY, HAVING y palabras reservadas como AS.	1	8
	T38	Implementar funcionalidad para traducción con agrupaciones y funciones de agregación.	1	8
QTE-03	T39	Extender gramática para cláusulas INNER JOIN, LEFT JOIN y RIGHT JOIN y la palabra reservada ON.	1	8
	T40	Implementar funcionalidad para traducción de consultas SELECT que incluyan relaciones.	1	8
QTE-04	T41	Extender gramática para sentencias INSERT INTO, UPDATE y DELETE.	1	5
	T42	Implementar funcionalidad para la traducción de INSERT INTO, UPDATE y DELETE.	1	5
QTE-05	T43	Implementar funcionalidad para la ejecución de consultas.	1	2
	T44	Implementar funcionalidad para visualización y formato de resultados.	1	5
	T45	Implementar funcionalidad para exportación de resultados en varios formatos.	2	5
	T46	Implementar endpoints para la ejecución de consultas y exportación de resultados.	1	5

Continúa en la siguiente página...

Cód. HU	Cód. Tarea	Tarea	Prioridad	Complejidad
	T47	Diseñar panel para resultados de la ejecución.	2	5
	T48	Desarrollar panel para visualización de resultados.	1	8
QTE-06	T49	Implementar funcionalidad para gestión de consultas almacenadas.	1	3
	T50	Implementar endpoints para la administración de consultas.	1	2
	T51	Diseñar pantalla de historial de consultas.	2	2
	T52	Desarrollar pantalla de historial de consultas.	1	3
OBS-01	T53	Implementar funcionalidad para almacenamiento de logs.	2	5
	T54	Implementar un endpoint para la obtención de logs.	2	2
	T55	Diseñar pantalla para visualización de logs.	3	3
	T56	Desarrollar pantalla para visualización de logs.	2	5
OBS-02	T57	Implementar funcionalidad para la obtención de estadísticas de uso del sistema.	3	5
	T58	Implementar un endpoint para la visualización de estadísticas globales del sistema.	3	3
	T59	Diseñar dashboard de estadísticas generales.	3	3
	T60	Desarrollar dashboard para la visualización de estadísticas generales.	3	8

Planificación de Sprints

Con el objetivo de organizar las tareas de la mejor manera para entregar un incremento funcional al final de cada iteración, se dividió el trabajo en 6 sprints, incluyendo un sprint 0 netamente relacionado con actividades de adecuar el editor de código y actividades relacionadas a la fase de diseño del ciclo de vida del desarrollo de software.

En la Tabla 2.9, se detalla la planificación de cada sprint, donde se indica la duración en semanas, los objetivos a cumplir y las tareas asignadas a cada uno de ellos.

Tabla 2.9: Planificación de Sprints.

Sprint	Duración	Objetivos	Tareas
Sprint 0	1 semana	■ Configurar el entorno de desarrollo. ■ Diseñar la arquitectura del sistema. ■ Diseñar la base de datos para gestión de usuarios, conexiones y consultas. ■ Prototipar las interfaces de usuario del sistema.	T05, T07, T12, T17, T21, T26, T30, T35, T47, T51, T55, T59
Sprint 1	2 semanas	■ Implementar el registro e inicio de sesión. ■ Implementar la configuración de conexiones.	T01, T02, T03, T04, T06, T08, T23, T24, T25, T27
Sprint 2	2 semanas	■ Implementar la traducción de sentencias SELECT estándar. ■ Implementar la ejecución de consultas traducidas.	T32, T33, T34, T36, T43, T44, T45, T46, T48
Sprint 3	2 semanas	■ Implementar la traducción de sentencias SELECT con cláusulas, funciones, operadores y relaciones. ■ Implementar la traducción de sentencias INSERT INTO, UPDATE y DELETE.	T37, T38, T39, T40, T41, T42

Continúa en la siguiente página...

Sprint	Duración	Objetivos	Tareas
Sprint 4	2 semanas	■ Implementar la modificación de perfil y restablecimiento de contraseña. ■ Implementar la gestión de usuarios y roles.	T09, T10, T11, T13, T14, T15, T16, T18, T19, T20, T22
Sprint 5	2 semanas	■ Implementar la gestión de conexiones. ■ Implementar el historial de consultas. ■ Implementar la visualización de logs.	T28, T29, T31, T49, T50, T52, T53, T54, T56
Sprint 6	2 semanas	■ Implementar el monitoreo de estadísticas globales del <i>middleware</i>.	T57, T58, T60

2.2.3. Sprint 0

Sprint 0 Planning

En la planificación del Sprint 0 se establecieron las tareas que formarán parte de esta primera iteración, mismas que serán guía para el desarrollo del sistema. El objetivo principal de cumplir con este backlog es la configuración del entorno de desarrollo, el diseño de la arquitectura del sistema y el prototipado de las interfaces de usuario.

Sprint 0 Backlog

A continuación, en la Tabla 2.10 se puede visualizar la lista de tareas correspondientes al Sprint 0.

Tabla 2.10: Sprint 0 Backlog.

Código	Tarea
T05	Diseñar pantalla de registro de usuario.
T07	Diseñar pantalla de inicio de sesión.
T12	Diseñar pantalla de actualización de perfil.
T17	Diseñar formularios para solicitar correo y restablecer contraseña.
T21	Diseñar pantalla de gestión de usuarios.

Continúa en la siguiente página...

Código	Tarea
T26	Diseñar modal para configurar una conexión.
T30	Diseñar pantalla de gestión de conexiones.
T35	Diseñar pantalla de traducción.
T47	Diseñar panel para resultados de la ejecución.
T51	Diseñar pantalla de historial de consultas.
T55	Diseñar pantalla para visualización de logs.
T59	Diseñar dashboard de estadísticas generales.

Ejecución del Sprint 0

En la ejecución de este sprint se llevaron a cabo las siguientes actividades clave:

Definición de arquitectura

La arquitectura de este middleware fue diseñada utilizando el modelo C4, un enfoque que permite representar la estructura del sistema a través de varios niveles de abstracción.

Diagrama de Contexto: La Figura 2.2 ilustra el primer nivel del modelo C4 donde se identifica la interacción con los usuarios y sistemas externos. Aquí se aprecian dos tipos de usuarios principales: el administrador del sistema, encargado de gestionar usuarios, supervisar el correcto uso de la aplicación, identificar mejoras en ella y, desarrolladores, principales interesados en utilizar las funciones de traducción y ejecución de consultas.

Por otro lado, se establecieron 3 sistemas externos dado que el middleware se conectará a los Sistemas Gestores de Bases de Datos SQL Server y Neo4j para acceder y manipular las bases de datos respectivas y un sistema que brinde el servicio de correo electrónico, necesario para el restablecimiento de contraseñas.

Diagrama de Contenedores: En el diagrama de contenedores de la Figura 2.3 se identifican los servicios que componen el sistema, la interacción entre ellos las tecnologías requeridas para su implementación. Dentro de los contenedores definidos se contempló la interfaz de usuario con la cual los usuarios interactuarán directamente, la api que contendrá la lógica de negocio, un motor de traducción que se encargará del mapeo de sentencias SQL a Cypher, un servicio de logs que registrará toda actividad ejecutada en el *middleware* y la base de datos interna, dedicada al almacenamiento de la información de usuarios,

conexiones y consultas.

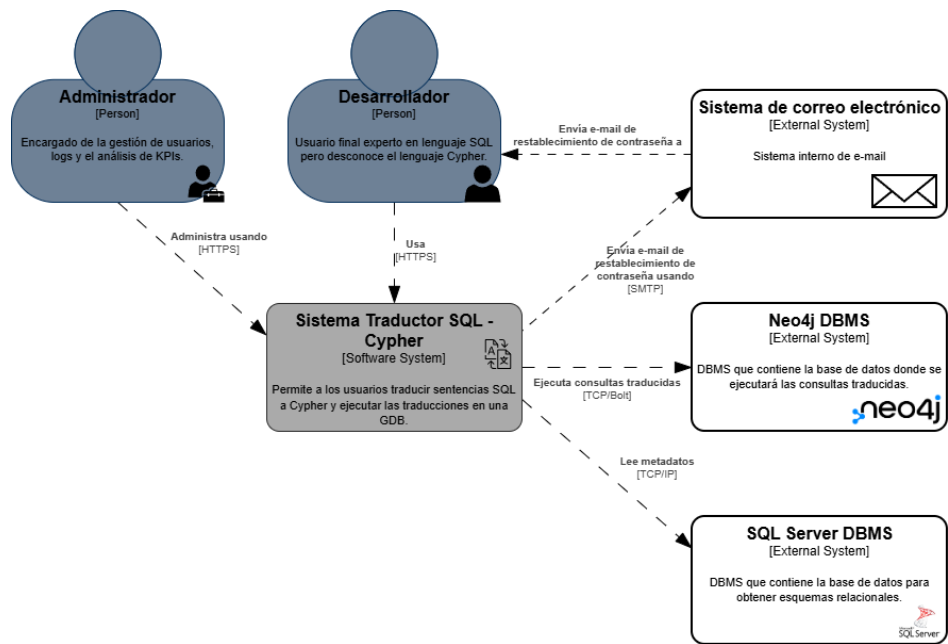


Figura 2.2: Diagrama de contexto del sistema.

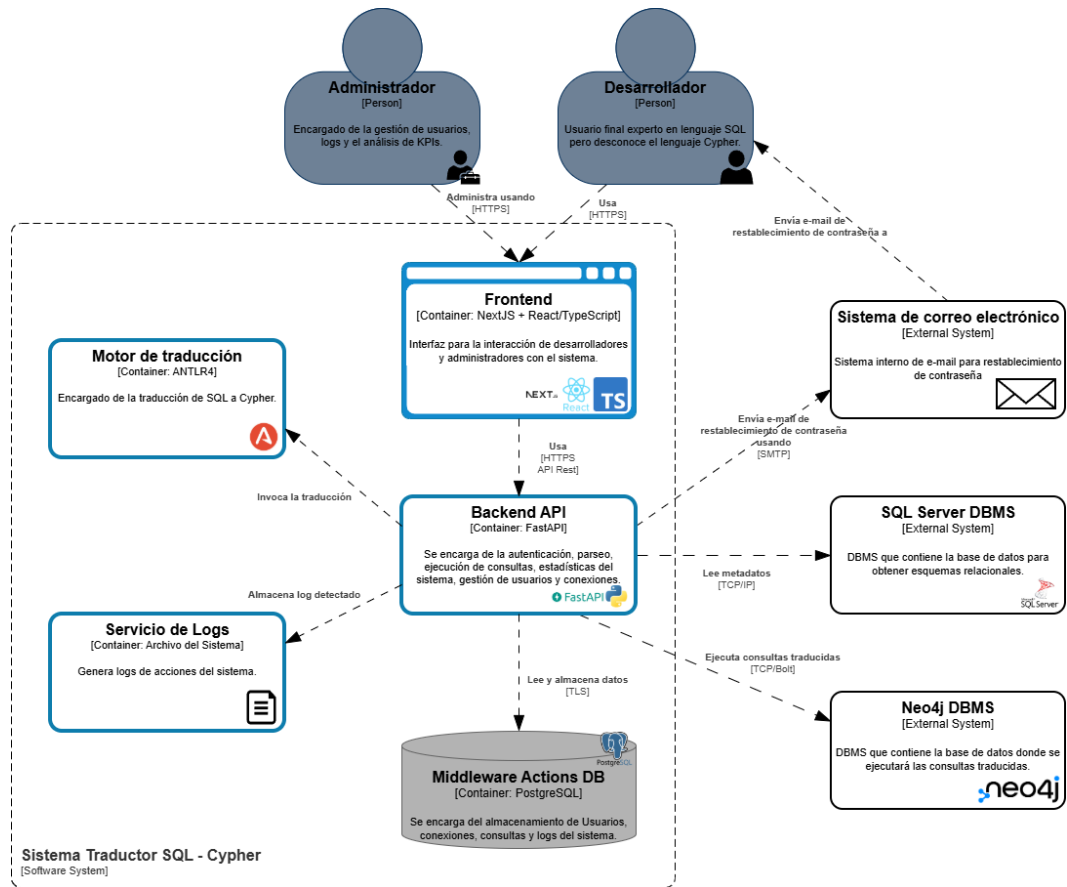


Figura 2.3: Diagrama de contenedores del sistema.

Definición de base de datos

De acuerdo a las Historias de Usuario definidas, se optó por implementar una base de datos relacional que contenga la información de los usuarios, la configuración de sus conexiones, las consultas realizadas y para poder dar seguimiento a cada una de las acciones que ejecuten sobre la aplicación. Esta base de datos que será implementada en PostgreSQL contiene 4 tablas principales: **USUARIOS**, **CONEXIONES**, **CONSULTAS** y **LOGS**, mismas que serán guías para garantizar la integridad de los datos. La Figura 2.4 ilustra el diagrama de base de datos propuesto con las tablas mencionadas, los campos que contendrán y las relaciones entre ellas.

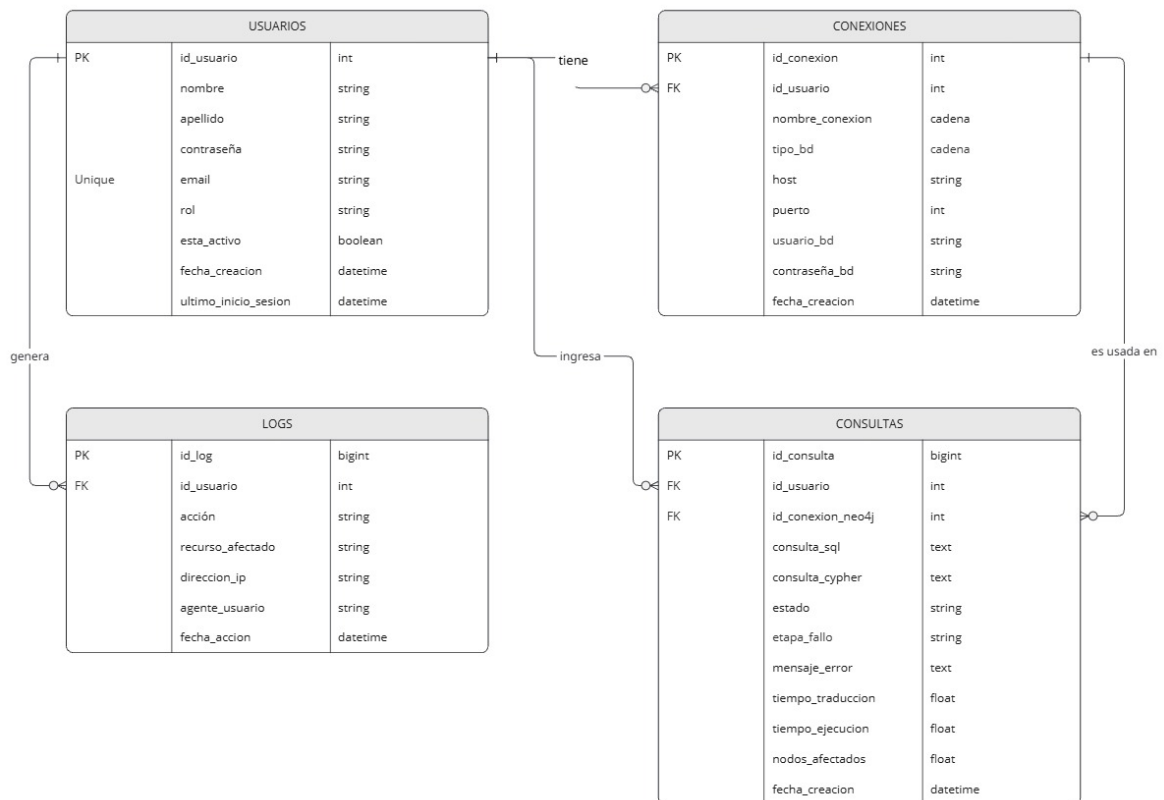


Figura 2.4: Diagrama de base de datos propuesta.

Diseño de prototipos de interfaces

Se realizaron prototipos de alta a través de Figma tal y como se puede observar en la Figura 2.5. Las pantallas diseñadas y que se encuentran en el Anexo II son las siguientes:

- Pantallas para desarrolladores y administradores
 - Inicio de sesión

- Registro de usuario
 - Dashboard principal
 - Editar perfil
 - Restablecimiento de contraseña
 - Configurar conexión
 - Gestión de conexiones
 - Traducción y ejecución de consultas
 - Historial de consultas
- Pantallas solo para administradores
 - Gestión de usuarios
 - Logs del sistema
 - Estadísticas del sistema

The image shows a Figma prototype of a web application interface, divided into two main sections: a registration/welcome screen on the left and a login screen on the right.

Left Section (Registration/Welcome):

- Header: **¡BIENVENIDO DE VUELTA!**
- Image: A 3D database cylinder icon.
- Text: **SQL2GRAPH**
- Text: ¿No tienes una cuenta?
- Button: **Regístrate** (outlined in blue)

Right Section (Login):

- Header: **INICIAR SESIÓN**
- Text: Correo electrónico
- Input field: `user@example.com`
- Text: Contraseña
- Input field: `*****`
- Button: **Iniciar Sesión** (solid dark blue)

Figura 2.5: Prototipo de pantalla de traducción realizado en Figma.

Configuración de herramientas de gestión del proyecto

Para supervisar un continuo progreso en el desarrollo del proyecto, se optó por 2 herramientas principales: Trello que se empleará para asignar las tareas establecidas al planificar los Sprints y conocer el estado de cada una dentro de su iteración. En la Figura 2.6 se puede observar la configuración para gestionar el proyecto, donde se crearon las columnas: Epics, Product Backlog, Sprint Backlog, In Progress, Code Review y Done. Las 3 primeras columnas servirán para identificar las Épicas e Historias de Usuario mencionadas en secciones anteriores y indicar cuáles se deberán completar en un determinado sprint. El resto de columnas indicarán el avance de cada Historia de Usuario.

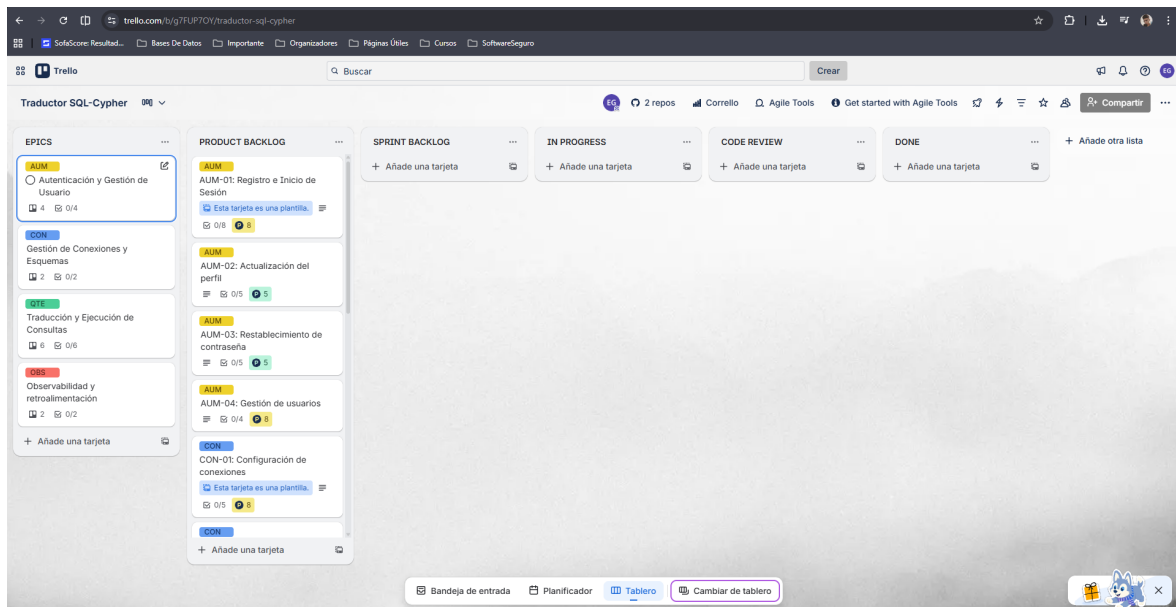


Figura 2.6: Configuración inicial del proyecto en Trello.

Para facilitar la comunicación entre todo el equipo de Scrum, se creó un canal en la plataforma Microsoft Teams donde se llevarán a cabo reuniones para revisar avances, discutir sobre los obstáculos que surjan a lo largo del proyecto y obtener constante retroalimentación por parte del director de tesis.

Configuración del entorno de desarrollo

Se creó una organización en GitHub que alberga 3 repositorios destinados para el backend, frontend y la documentación del sistema. Esto se puede observar en la Figura 2.7.

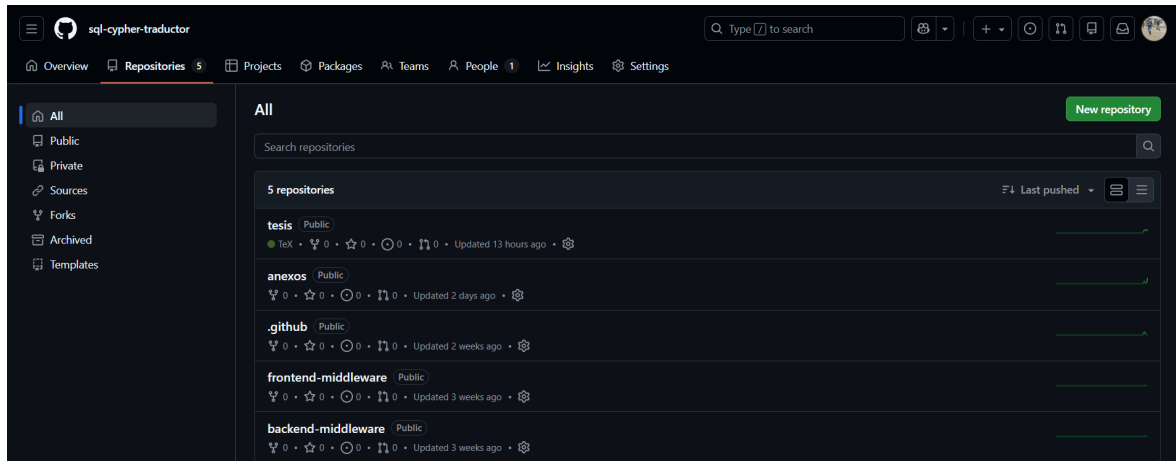


Figura 2.7: Distribución de repositorios en GitHub.

Para el desarrollo del frontend y backend se estableció un flujo de trabajo con 2 ramas principales: `main` que tendrá el código funcional y `develop` que será utilizada como una rama de integración donde se implementarán, testearán, corregirán e integrarán las funcionalidades del *middleware*.

Sprint 0 Review

Durante este sprint se completaron con todas las tareas que fueron incluidas en el backlog, además de diseñar la arquitectura del sistema, la base de datos interno y la configuración de las plataformas para gestión del proyecto e implementación de código. Al cumplir con todas las actividades antes mencionadas, se iniciará con el desarrollo de código del sistema a partir del Sprint 1.

Sprint 0 Retrospective

■ ¿Qué salió bien?

El diseño de la arquitectura del sistema, las bases de datos y los prototipos de interfaces de usuario en conjunto con el stack tecnológico brindó una orientación para estructurar los repositorios y que puedan ser escalables y mantenibles.

■ ¿Qué se puede mejorar?

La falta de conocimiento en el uso de Trello retrasó a la configuración inicial de la herramienta que brinda una variedad de herramientas que optimicen la gestión de proyectos y la integración con GitHub. Se propone una capacitación de esta platafor-

ma para sacar provecho de esta aplicación y aumentar la productividad del equipo.

2.3. Fase de Desarrollo

En esta fase se detalla todo el desarrollo del *middleware* para traducir sentencias SQL a su equivalente en Cypher y su ejecución en una base de datos orientada a grafos. Para una mejor comprensión por parte del lector, cada uno de los 6 sprints establecidos presentan la siguiente estructura: Sprint Planning, Sprint Backlog, Ejecución del Sprint, Sprint Review y Sprint Retrospective.

2.3.1. Sprint 1

Sprint 1 Planning

El objetivo principal del Sprint 1 fue implementar el servicio de autenticación y la conexión a los DBMS tanto de SQL Server como de Neo4j. Con esto se busca establecer las funciones de acceso al sistema y configurar la conexión a las bases de datos para acceder e interactuar con ellas.

Sprint 1 Backlog

La Tabla 2.11 detalla las tareas del Backlog para esta primera iteración, mismas que abordan las Historias de Usuario AUM-01 y CON-01.

Tabla 2.11: Sprint 1 Backlog.

Código	Tarea
T01	Implementar funcionalidad para registro de usuario.
T02	Implementar funcionalidad para inicio de sesión.
T03	Implementar validación de campos y credenciales para autenticación.
T04	Implementar endpoints para registro e inicio de sesión.
T06	Desarrollar pantalla de registro de usuario.
T08	Desarrollar pantalla de inicio de sesión.
T23	Implementar funcionalidad para configurar las conexiones a las bases de datos.

Continúa en la siguiente página...

Código	Tarea
T24	Implementar pruebas de conectividad y funciones de conexión / desconexión.
T25	Implementar un endpoint para el almacenamiento de una conexión.
T27	Desarrollar modal para configurar una conexión.

Ejecución del Sprint 1

Este apartado lo dividiremos en 2 partes: el desarrollo del backend y la implementación del frontend.

Backend

Implementación de modelos y DTOs: Para implementar la autenticación de usuarios y gestión de conexiones, se procedió a definir los modelos de datos necesarios utilizando la biblioteca SQLAlchemy, misma que permite gestionar la información contenida en la base de datos del sistema garantizando su seguridad e integridad.

A continuación, se presenta un fragmento de los modelos principales `User` y `DbConnection` necesarios para la gestión de usuarios y conexiones respectivamente.

```

1      # Modelo de datos para Usuario
      class User(Base):
          __tablename__ = "users"

5          user_id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4,
                               index=True)
          email = Column(String, unique=True, index=True, nullable=False)
          full_name = Column(String, nullable=True)
          hashed_password = Column(String, nullable=False)
          is_active = Column(Boolean, default=True)
10         created_at = Column(DateTime(timezone=True), server_default=func.now())

          connections = relationship("DbConnection", back_populates="owner")

      # Modelo de datos para Conexiones a BD
15     class DbConnection(Base):
          __tablename__ = "db_connections"

```

```

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4, index=True)
    alias = Column(String, nullable=False)
    engine = Column(String, nullable=False)
    host = Column(String, nullable=False)
    port = Column(String, nullable=False)
    username = Column(String, nullable=False)
    encrypted_password = Column(String, nullable=False)
    db_name = Column(String, nullable=True)
    user_id = Column(UUID(as_uuid=True), ForeignKey("users.id"))

    owner = relationship("User", back_populates="connections")

```

Una vez implementada la estructura de modelos y utilizando la biblioteca Pydantic, se definieron los DTOs para validar qué datos se ingresarán al sistema y a cuáles tendrán acceso al usuario. Se realizaron DTOs tanto para usuarios, como para conexiones y además para el envío de tokens, mismos que serán utilizados para el inicio de sesión del cual se tratará más adelante. En la parte inferior se puede visualizar un extracto de código correspondiente a los DTOs para la gestión de los usuarios.

```

1  class UserDTO(BaseModel):
    email: EmailStr

    class UserCreatedDTO(UserDTO):
5      password: str = Field(..., min_length=6)
      full_name: Optional[str] = None

    class UserResponseDTO(UserDTO):
10      id: UUID
      full_name: Optional[str] = None
      is_active: bool
      created_at: datetime

    class Config:
15      from_attributes = True

```

Cifrado de contraseñas y manejo de sesiones En el servicio de autenticación, se implementó el algoritmo Bcrypt para hashear la contraseña del usuario previo a ser almacenada.

```

1 def get_password_hash(password):
    pwd_bytes = password.encode("utf-8")
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(pwd_bytes, salt)
5 return hashed.decode("utf-8")

```

Por su parte, para el manejo de sesiones se utilizó JWT, mismo que genera tokens que contiene información del usuario cifrada y un tiempo de expiración. El tiempo de caducidad del token está configurado para 15 minutos y la firma para generación de tokens se obtiene de las variables de entorno con lo cual se refuerza la seguridad del software.

```

1 def create_access_token(data: dict, expires_delta: Optional[timedelta]):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now() + expires_delta
5    else:
        expire = datetime.now() + timedelta(minutes=15)

    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, JWT_SECRET_KEY, algorithm=ALGORITHM)
10 return encoded_jwt

```

Para el almacenamiento de conexiones a bases de datos, se utilizó el sistema Fernet, que se encargará de encriptar las contraseñas de las BD previo a ser almacenadas.

```

1 def encrypt_credential(raw_password: str) -> str:
    return cipher_suite.encrypt(raw_password.encode()).decode()

```

Especificación de Endpoints En este sprint, se implementaron las peticiones HTTP asociadas a las funcionalidades de autenticación y conexión a base de datos. En la Tabla 2.12 se detalla cada uno de los endpoints desarrollados y la función que cumplen.

Tabla 2.12: Endpoints de autenticación y gestión de conexiones

Método HTTP	Endpoint	Descripción
GET	/connections	Indica todas las conexiones del usuario.
GET	/connections/conn-id	Indica los detalles de una conexión específica.
POST	/connections	Agrega una nueva conexión.
PUT	/connections/conn-id	Actualiza una conexión existente.
DELETE	/connections/conn-id	Elimina una conexión
POST	/connections/conn-id/test	Permite realizar pruebas de conexión.
POST	/auth/register	Permite registrar un nuevo usuario.
POST	/auth/login	Permite iniciar sesión y solicita la generación un token JWT.

Frontend:

Aquí se construyeron las pantallas respectivas para registro y login. Se utilizó la librería Zod para validar los campos desde el lado del cliente asegurando que cumplan los formatos correctos previo a la integración con el backend. La gestión del estado de autenticación se maneja con Zustand, almacenando el token de acceso y la información del usuario de forma segura.

```

1  "use client";
   import { useAuthStore } from "@store/useAuthStore";
   // ...

5  export default function LoginPage() {
   const { login, isLoading, error } = useAuthStore();
   // ...

   const onSubmit = async (values: LoginData) => {
       await login(values);

10  };
   // ...
   }

```

Además, se implementó un modal en la interfaz de usuario donde los usuarios deben ingresar los detalles de la conexión (host, puerto, usuario, contraseña, etc.). Este modal está integrado directamente con la lógica de configuración de conexiones implementada en el backend y maneja una buena retroalimentación que indica al usuario de forma amigable si existen o no problemas de conexión.

```
1  "use client";
   import { useConnectionStore } from "@store/useConnectionStore";
   // ...

5  export function ConnectionModal() {
   const { createConnection, loading } = useConnectionStore();
   // ...

   const onSubmit = async (data: ConnectionFormData) => {
       const success = await createConnection(data);
10      if (success) {
           // ...
       }
   };
   // ...

15 }
```

Sprint 1 Review

Al finalizar el Sprint 1, se completaron con éxito las tareas planificadas en el backlog. El sistema ahora cuenta con un servicio de autenticación funcional que permite a los usuarios registrarse e iniciar sesión de forma segura. Asimismo, la aplicación brinda la facilidad para configurar y almacenar las credenciales de sus bases de datos SQL Server y Neo4j. A través de las pruebas unitarias y de integración realizadas se asegura la calidad y estabilidad de la implementación en esta primera iteración.

Sprint 1 Retrospective

■ ¿Qué salió bien?

La implementación de los flujos de autenticación y conexión se realizó sin mayores contratiempos. La separación de responsabilidades entre el frontend y el backend,

junto con el uso de FastAPI y Next.js, demostró ser una arquitectura robusta y escalable. La gestión de estado con Zustand en el frontend simplificó el manejo de la información del usuario y las conexiones.

■ ¿Qué se puede mejorar?

Aunque la funcionalidad principal está implementada, se podría mejorar la experiencia de usuario añadiendo retroalimentación más detallada durante los procesos de conexión. Para el backend, se podría considerar la implementación de un sistema de refresco de tokens (refresh tokens) para mejorar la seguridad y la persistencia de la sesión del usuario. Además, sería beneficioso ampliar la cobertura de las pruebas para incluir más casos de borde.

2.3.2. Sprint 2

Sprint 2 Planning

El objetivo de este sprint fue implementar la funcionalidad principal del traductor, permitiendo a los usuarios convertir sentencias `SELECT` estándar de SQL a su equivalente en Cypher. Adicionalmente, se buscó habilitar la ejecución de estas consultas traducidas en la base de datos Neo4j y visualizar los resultados obtenidos.

Sprint 2 Backlog

La Tabla 2.13 detalla las tareas del Sprint 2.

Tabla 2.13: Sprint 2 Backlog.

Código	Tarea
T32	Definir reglas léxicas y sintácticas de la sentencia <code>SELECT</code> , las cláusulas <code>WHERE</code> , <code>ORDER BY</code> , palabras reservadas como <code>FROM</code> , <code>DISTINCT</code> , <code>LIMIT</code> , operadores lógicos y de comparación.
T33	Implementar funcionalidad para traducción de consultas <code>SELECT</code> .
T34	Implementar un endpoint para la traducción de consultas.
T36	Desarrollar pantalla de traducción de consultas.
T43	Implementar funcionalidad para la ejecución de consultas.

Continúa en la siguiente página...

Código	Tarea
T44	Implementar funcionalidad para visualización y formato de resultados.
T45	Implementar funcionalidad para exportación de resultados en varios formatos.
T46	Implementar endpoints para la ejecución de consultas y exportación de resultados.
T48	Desarrollar panel para visualización de resultados.

Ejecución del Sprint 2

El trabajo de este sprint se centró en dos áreas principales: la traducción de sentencias `SELECT` y la ejecución y visualización de las consultas resultantes.

Traducción de Consultas `SELECT`

Backend: Se utilizó ANTLR (Another Tool for Language Recognition) para construir el parser del dialecto SQL soportado. Se definieron las reglas léxicas y sintácticas en un archivo de gramática (`SQLSimple.g4`) para reconocer la estructura de una sentencia `SELECT`, incluyendo cláusulas como `WHERE` y `ORDER BY`, y palabras clave como `DISTINCT` y `LIMIT`.

A partir de esta gramática, se generaron las clases del lexer, parser y un visitor. Se implementó una clase `Visitor` personalizada que recorre el árbol sintáctico abstracto (AST) generado por el parser y construye la consulta Cypher equivalente. Finalmente, se expuso esta lógica a través de un endpoint `/translate`.

```

1 # app/core/parser/visitor.py
class Visitor(SQLSimpleVisitor):
    def visitSelect_statement(self, ctx: SQLSimpleParser.Select_statementContext)
      :
      # ... Logica de traduccion ...
5     cypher_query = f"MATCH {match_clause} RETURN {return_clause}"
      # ...
      return cypher_query

```

Figura 2.8: Implementación del Visitor para la traducción.

Frontend: Se desarrolló la pantalla principal de traducción, que consta de dos editores de código: uno para ingresar la consulta SQL (SQLEditor) y otro para mostrar la consulta Cypher resultante (CypherViewer). Se utilizó la librería CodeMirror para crear estos editores, proporcionando resaltado de sintaxis y una mejor experiencia de usuario. La comunicación con el endpoint de traducción del backend se gestionó a través de un store de Zustand (useTranslateStore).

```
1 // src/app/dashboard/translate/page.tsx
  "use client";
  import { SQLEditor } from "@components/translator/SQLEditor";
  import { CypherViewer } from "@components/translator/CypherViewer";
5  import { useTranslateStore } from "@store/useTranslateStore";

  export default function TranslatePage() {
    const { sqlQuery, setSqlQuery, translatedQuery, translateQuery, loading } =
      useTranslateStore();

10
    const handleTranslate = async () => {
      await translateQuery(sqlQuery);
    };

15
    return (
      // ...
      <SQLEditor value={sqlQuery} onChange={setSqlQuery} />
      <button onClick={handleTranslate}>Translate </button>
      <CypherViewer value={translatedQuery} />
20      // ...
    );
  }
```

Figura 2.9: Componente de la página de traducción.

Ejecución y Visualización de Consultas

Backend: Se crearon los endpoints necesarios para ejecutar las consultas Cypher en la base de datos Neo4j configurada por el usuario y para exportar los resultados. El servicio de ejecución se conecta a Neo4j, ejecuta la consulta y devuelve los resultados en formato JSON.

Frontend: Se implementó un panel de resultados que muestra los datos obtenidos de la ejecución de la consulta en una tabla. Este panel también incluye la funcionalidad para exportar los resultados en formatos como CSV y JSON, permitiendo al usuario trabajar con los datos fuera de la aplicación.

Sprint 2 Review

Al concluir el Sprint 2, se logró el objetivo de implementar la traducción y ejecución de consultas `SELECT` simples. Los usuarios ahora pueden escribir una consulta SQL, obtener su equivalente en Cypher y ejecutarla contra su base de datos Neo4j, todo desde una única interfaz. La visualización de resultados en formato de tabla y las opciones de exportación añaden un valor significativo a la herramienta.

Sprint 2 Retrospective

■ **¿Qué salió bien?**

La integración de ANTLR para el parsing de SQL fue un éxito y sentó una base sólida para futuras extensiones de la gramática. La arquitectura del frontend, utilizando componentes reutilizables y un store centralizado, facilitó el desarrollo de una interfaz de usuario interactiva y cohesiva.

■ **¿Qué se puede mejorar?**

El manejo de errores en la traducción puede ser más robusto. Actualmente, si una consulta SQL no es válida, el sistema podría no proporcionar una retroalimentación clara al usuario. Se debería mejorar el visitor de ANTLR para capturar y reportar errores de sintaxis de manera más efectiva. Además, la visualización de resultados podría enriquecerse para incluir una vista de grafo, lo cual sería más intuitivo para una base de datos como Neo4j.

2.3.3. Sprint 3

Sprint 3 Planning

El objetivo de este sprint fue ampliar significativamente las capacidades del traductor SQL a Cypher. Se planificó extender la gramática ANTLR para soportar funcionalidades más avanzadas de SQL, incluyendo funciones de agregación (`COUNT`, `SUM`, etc.), agrupaciones

(`GROUP BY`, `HAVING`), relaciones entre tablas (`JOIN`) y operaciones de manipulación de datos (`INSERT`, `UPDATE`, `DELETE`).

Sprint 3 Backlog

La Tabla 2.14 detalla las tareas del Sprint 3.

Tabla 2.14: Sprint 3 Backlog.

Código	Tarea
T37	Extender gramática para funciones de agregación como <code>COUNT</code> , <code>SUM</code> , <code>MIN</code> , <code>MAX</code> o <code>AVG</code> , cláusulas <code>ORDER BY</code> , <code>HAVING</code> y palabras reservadas como <code>AS</code> .
T38	Implementar funcionalidad para traducción con agrupaciones y funciones de agregación.
T39	Extender gramática para cláusulas <code>INNER JOIN</code> , <code>LEFT JOIN</code> y <code>RIGHT JOIN</code> y la palabra reservada <code>ON</code> .
T40	Implementar funcionalidad para traducción de consultas <code>SELECT</code> que incluyan relaciones.
T41	Extender gramática para sentencias <code>INSERT INTO</code> , <code>UPDATE</code> y <code>DELETE</code> .
T42	Implementar funcionalidad para la traducción de <code>INSERT INTO</code> , <code>UPDATE</code> y <code>DELETE</code> .

Ejecución del Sprint 3

El desarrollo se enfocó exclusivamente en el backend, específicamente en la mejora del parser y el visitor de ANTLR para dar soporte a las nuevas características del lenguaje SQL.

Extensión de la Gramática y Lógica de Traducción

Funciones de Agregación y Agrupación: Se modificó el archivo de gramática `SQLSimple.g4` para incluir reglas que reconocen funciones de agregación y las cláusulas `GROUP BY` y `HAVING`. La clase `Visitor` fue actualizada para traducir estas construcciones a sus equivalentes en Cypher. Por ejemplo, una función `COUNT(columna)` se traduce a `count(n.columna)` y la cláusula `GROUP BY` se mapea a la cláusula `WITH` en Cypher para agrupar los resultados.

Traducción de Relaciones (JOINS): Se añadieron reglas a la gramática para los operadores `INNER JOIN`, `LEFT JOIN` y `RIGHT JOIN`. La lógica en el `Visitor` se extendió para

interpretar estas uniones y generar las cláusulas `MATCH` correspondientes en Cypher que representan las relaciones entre nodos. Por ejemplo, un `INNER JOIN` entre dos tablas se traduce a un patrón `MATCH` que conecta dos tipos de nodos.

Traducción de DML (INSERT, UPDATE, DELETE): Finalmente, se extendió la gramática para soportar las sentencias de manipulación de datos.

- `INSERT INTO`: Se traduce a una sentencia `CREATE` en Cypher para crear un nuevo nodo con las propiedades especificadas.
- `UPDATE`: Se mapea a una combinación de `MATCH` y `SET` para encontrar los nodos a actualizar y modificar sus propiedades.
- `DELETE`: Se convierte en una sentencia `MATCH` seguida de `DETACH DELETE` para eliminar los nodos y sus relaciones asociadas.

```
1 # app/core/parser/visitor.py
class Visitor(SQLSimpleVisitor):
    # ... (metodos existentes) ...

5     def visitInsert_statement(self, ctx: SQLSimpleParser.Insert_statementContext)
        :
        # Logica para traducir INSERT a CREATE
        return "CREATE (n:Label {key: 'value'})"

    def visitUpdate_statement(self, ctx: SQLSimpleParser.Update_statementContext)
        :
10     # Logica para traducir UPDATE a MATCH y SET
        return "MATCH (n:Label WHERE n.id = 1) SET n.property = 'new_value'"

    def visitDelete_statement(self, ctx: SQLSimpleParser.Delete_statementContext)
        :
15     # Logica para traducir DELETE a MATCH y DETACH DELETE
        return "MATCH (n:Label WHERE n.id = 1) DETACH DELETE n"
```

Figura 2.10: Nuevos métodos en el Visitor para DML.

Sprint 3 Review

Al finalizar el Sprint 3, el traductor SQL a Cypher es considerablemente más potente. Ahora puede manejar una gama mucho más amplia de consultas SQL, incluyendo agregaciones, agrupaciones, uniones y operaciones de modificación de datos. Esto representa un hito importante en el proyecto, ya que la funcionalidad principal ha alcanzado un alto grado de madurez. Todas las nuevas reglas de traducción fueron validadas mediante pruebas unitarias que cubren diversos escenarios y casos de borde.

Sprint 3 Retrospective

- **¿Qué salió bien?**

La base establecida con ANTLR en el sprint anterior demostró ser muy flexible y extensible, lo que permitió añadir las nuevas funcionalidades de manera estructurada y eficiente. El enfoque de desarrollo guiado por pruebas (TDD) fue clave para asegurar que las nuevas reglas de traducción funcionaran correctamente y no introdujeran regresiones en la funcionalidad existente.

- **¿Qué se puede mejorar?**

La complejidad del `Visitor` ha aumentado significativamente. Sería beneficioso refactorizar esta clase para separar la lógica de traducción de las diferentes tipos de sentencias SQL en módulos o clases más pequeñas y especializadas. Esto mejoraría la mantenibilidad y legibilidad del código. Además, la traducción de `JOIN` complejos con múltiples condiciones podría optimizarse para generar patrones de Cypher más eficientes.

2.3.4. Sprint 4

Sprint 4 Planning

El objetivo de este sprint fue mejorar la gestión de usuarios y la seguridad de la aplicación. Se planificó implementar la funcionalidad de actualización de perfil, el restablecimiento de contraseña y la administración de usuarios por parte de un administrador.

Sprint 4 Backlog

La Tabla 2.15 detalla las tareas del Sprint 4.

Tabla 2.15: Sprint 4 Backlog.

Código	Tarea
T09	Implementar funcionalidad para actualización de perfil.
T10	Implementar validación de campos modificados.
T11	Implementar un endpoint para actualizar información.
T13	Desarrollar pantalla de actualización de datos.
T14	Implementar funcionalidad para restablecimiento de contraseña.
T15	Implementar validación de email asociado a una cuenta.
T16	Implementar validación de formato para nueva contraseña.
T18	Desarrollar formularios para solicitud de correo y restablecimiento de contraseña.
T19	Implementar funcionalidad para gestión de usuarios.
T20	Implementar endpoints correspondientes para administración de usuarios.
T22	Desarrollar pantalla de gestión de usuarios.

Ejecución del Sprint 4

El desarrollo se distribuyó entre el backend y el frontend para implementar el flujo completo de las nuevas funcionalidades de gestión de usuarios.

Actualización de Perfil y Restablecimiento de Contraseña

Backend: Se crearon nuevos endpoints para permitir a los usuarios actualizar su información de perfil (nombre, apellido) y para manejar el proceso de restablecimiento de contraseña. El flujo de restablecimiento de contraseña incluye:

1. Un endpoint que recibe el correo electrónico del usuario, genera un token de restablecimiento único y lo envía por correo electrónico.
2. Un endpoint que valida el token recibido y permite al usuario establecer una nueva contraseña.

Se implementó un servicio de correo electrónico para enviar el enlace de restablecimiento.

```
1 # app/api/v1/endpoints/auth.py
@router.post("/password-recovery/{email}", response_model=Msg)
def recover_password(email: str, db: Session = Depends(deps.get_db)) -> Any:
    """
5     Password Recovery
    """
    user = auth_service.get_user_by_email(db, email=email)

    if not user:
10         raise HTTPException(
            status_code=404,
            detail="The user with this email does not exist in the system.",
        )
    password_reset_token = password_reset_service.create_token(db, user_id=user.id)
15    email_service.send_reset_password_email(
        email_to=user.email, username=user.first_name, token=password_reset_token.token
    )
    return {"msg": "Password recovery email sent"}
```

Figura 2.11: Endpoint para solicitar restablecimiento de contraseña.

Frontend: Se desarrollaron las pantallas correspondientes en el frontend:

- Una página de "Editar Perfil" donde los usuarios pueden modificar su información.
- Un formulario para solicitar el correo de restablecimiento de contraseña.
- Una página que recibe el token de la URL y permite al usuario ingresar su nueva contraseña.

Se añadieron las validaciones de formulario necesarias para asegurar, por ejemplo, que la nueva contraseña cumpla con los requisitos de seguridad.

Gestión de Usuarios (Admin)

Backend: Se implementaron endpoints protegidos que solo pueden ser accedidos por usuarios con rol de "administrador". Estos endpoints permiten realizar operaciones CRUD

(Crear, Leer, Actualizar, Eliminar) sobre los usuarios del sistema.

Frontend: Se creó una nueva sección en el dashboard, visible solo para administradores, que muestra una tabla con todos los usuarios registrados. Desde esta pantalla, un administrador puede ver, editar y eliminar usuarios, así como asignar roles.

Sprint 4 Review

Al finalizar el Sprint 4, la aplicación cuenta con un conjunto completo de herramientas para la gestión de cuentas de usuario. Los usuarios pueden gestionar su propia información y recuperar el acceso a sus cuentas si olvidan la contraseña. Los administradores tienen ahora la capacidad de supervisar y administrar a todos los usuarios del sistema. Estas características mejoran significativamente la usabilidad y la seguridad de la plataforma.

Sprint 4 Retrospective

■ **¿Qué salió bien?**

La implementación del flujo de restablecimiento de contraseña, que involucra la generación de tokens, el envío de correos electrónicos y la validación, se completó con éxito y funciona de manera robusta. La protección de rutas y endpoints basada en roles se implementó correctamente, asegurando que solo los usuarios autorizados puedan acceder a las funciones de administración.

■ **¿Qué se puede mejorar?**

El sistema de envío de correos electrónicos es funcional, pero podría beneficiarse del uso de plantillas HTML para que los correos sean más atractivos y profesionales. En la gestión de usuarios, se podría añadir una funcionalidad de búsqueda y filtrado para que los administradores puedan encontrar usuarios más fácilmente. La auditoría de acciones de administrador (quién hizo qué y cuándo) sería una adición valiosa para la seguridad y el monitoreo.

2.3.5. Sprint 5

Sprint 5 Planning

El objetivo de este sprint fue introducir funcionalidades para la gestión y el monitoreo de la actividad en la aplicación. Se planificó implementar la administración de conexiones guardadas, un historial de consultas ejecutadas y un sistema de visualización de logs de actividad.

Sprint 5 Backlog

La Tabla 2.16 detalla las tareas del Sprint 5.

Tabla 2.16: Sprint 5 Backlog.

Código	Tarea
T28	Implementar funcionalidad para administración de conexiones.
T29	Implementar endpoints para la gestión de conexiones.
T31	Desarrollar pantalla para administrar conexiones.
T49	Implementar funcionalidad para gestión de consultas almacenadas.
T50	Implementar endpoints para la administración de consultas.
T52	Desarrollar pantalla de historial de consultas.
T53	Implementar funcionalidad para almacenamiento de logs.
T54	Implementar un endpoint para la obtención de logs.
T56	Desarrollar pantalla para visualización de logs.

Ejecución del Sprint 5

El desarrollo se centró en crear nuevas secciones en el backend y frontend para que los usuarios puedan gestionar sus recursos y para que los administradores puedan monitorear la actividad del sistema.

Gestión de Conexiones

Backend: Se expandió la funcionalidad de conexiones para permitir operaciones CRUD completas. Se crearon endpoints para que los usuarios puedan listar, actualizar y eliminar sus conexiones a bases de datos guardadas.

Frontend: Se desarrolló una nueva página en el dashboard que muestra las conexiones guardadas por el usuario en forma de tarjetas. Cada tarjeta muestra los detalles de la conexión y ofrece botones para editarla o eliminarla.

```
1 // src/app/dashboard/connections/page.tsx
  "use client";
  import { ConnectionCard } from "@/components/connections/ConnectionCard";
  import { useConnectionStore } from "@/store/useConnectionStore";
5 // ...

  export default function ConnectionsPage() {
    const { connections, fetchConnections } = useConnectionStore();

10    useEffect(() => {
      fetchConnections();
    }, [fetchConnections]);

    return (
15      // ...
      {connections.map((connection) => (
        <ConnectionCard key={connection.id} connection={connection} />
      ))}
      // ...
20    );
  }
```

Figura 2.12: Página de gestión de conexiones.

Historial de Consultas

Backend: Se implementó un mecanismo para guardar cada consulta que un usuario traduce y/o ejecuta. Se creó un nuevo modelo en la base de datos (Query) y un endpoint para que los usuarios puedan recuperar su historial de consultas.

Frontend: Se desarrolló una pantalla de "Historial" que muestra una lista de las consultas realizadas por el usuario. Cada elemento de la lista muestra la consulta SQL original, la consulta Cypher traducida y la fecha en que se realizó. Esto permite a los usuarios reutilizar consultas anteriores fácilmente.

Visualización de Logs

Backend: Se configuró un sistema de logging para registrar eventos importantes de la aplicación, como inicios de sesión, errores de traducción o consultas ejecutadas. Se creó un endpoint, accesible solo para administradores, que permite recuperar estos logs.

Frontend: Se desarrolló una pantalla de "Logs" en la sección de administración. Esta pantalla muestra los logs en un formato claro y estructurado, permitiendo a los administradores monitorear la salud y el uso del sistema en tiempo real.

Sprint 5 Review

Al finalizar el Sprint 5, la aplicación ha ganado importantes capacidades de gestión y monitoreo. Los usuarios ahora tienen un control total sobre sus conexiones guardadas y pueden acceder a su historial de trabajo. Los administradores disponen de una herramienta de logging para supervisar la actividad del sistema. Estas características hacen que la aplicación sea más robusta, usable y administrable.

Sprint 5 Retrospective

■ ¿Qué salió bien?

La implementación de las funcionalidades CRUD para conexiones y el historial de consultas se realizó de manera fluida. La arquitectura existente facilitó la adición de nuevos modelos y endpoints. El sistema de logging proporciona información valiosa para la depuración y el monitoreo.

■ ¿Qué se puede mejorar?

La visualización de logs es funcional, pero podría mejorarse con capacidades de filtrado y búsqueda por nivel de severidad, usuario o rango de fechas. Para el historial de consultas, sería útil añadir una función para "marcar como favorito" las consultas más utilizadas. En la gestión de conexiones, se podría añadir un indicador visual del estado de la conexión (activa/inactiva) en tiempo real.

2.3.6. Sprint 6

Sprint 6 Planning

El objetivo del último sprint fue implementar un dashboard de estadísticas para proporcionar una visión general del uso y rendimiento del sistema. Esta funcionalidad está dirigida principalmente a los administradores para que puedan monitorear la actividad de la plataforma.

Sprint 6 Backlog

La Tabla 2.17 detalla las tareas del Sprint 6.

Tabla 2.17: Sprint 6 Backlog.

Código	Tarea
T57	Implementar funcionalidad para la obtención de estadísticas de uso del sistema.
T58	Implementar un endpoint para la visualización de estadísticas globales del sistema.
T60	Desarrollar dashboard para la visualización de estadísticas generales.

Ejecución del Sprint 6

El trabajo se concentró en la recopilación de datos en el backend y la presentación de estos datos de forma visual en el frontend.

Obtención y Exposición de Estadísticas

Backend: Se desarrolló un nuevo servicio de analíticas (`analytics_service.py`) encargado de calcular diversas métricas del sistema. Este servicio realiza consultas a la base de datos para obtener datos como:

- Número total de usuarios registrados.
- Número de consultas traducidas.
- Tipos de sentencias SQL más comunes (`SELECT`, `INSERT`, etc.).

- Actividad de los usuarios a lo largo del tiempo.

Se creó un endpoint `/analytics` protegido, accesible solo para administradores, que devuelve estas estadísticas en formato JSON.

```
1 # app/api/v1/endpoints/analytics.py
  @router.get("/", response_model=AnalyticsData)
  def get_analytics_data(
      db: Session = Depends(deps.get_db),
5      current_user: User = Depends(deps.get_current_active_admin),
  ) -> Any:
      """
      Retrieve analytics data.
      """
10     analytics_data = analytics_service.get_analytics(db)
    return analytics_data
```

Figura 2.13: Endpoint para obtener datos de analítica.

Dashboard de Estadísticas

Frontend: Se desarrolló una nueva página de "Dashboard" en la sección de administración. Esta página consume los datos del endpoint `/analytics` y los presenta de manera visual utilizando gráficos y tarjetas informativas. Se utilizó una librería de gráficos como Chart.js (a través de un wrapper para React) para crear visualizaciones como:

- Tarjetas que muestran el total de usuarios, traducciones y conexiones.
- Un gráfico de barras mostrando la distribución de los tipos de sentencias SQL.
- Un gráfico de líneas mostrando el número de registros de nuevos usuarios a lo largo del tiempo.

```

1 // src/app/dashboard/admin/page.tsx
  "use client";
  // ... imports para componentes de graficos y store ...

5 export default function AdminDashboardPage() {
  // ... Logica para obtener los datos de analitica del store ...

  return (
    <div>
10     <h1>Admin Dashboard</h1>
      { /* Tarjetas con estadisticas principales */ }
      <div className="grid grid-cols-3 gap-4">
        <Card title="Total Users" value={analytics.totalUsers} />
        <Card title="Total Translations" value={analytics.totalTranslations} />
15     { /* ... mas tarjetas ... */ }
      </div>

      { /* Graficos */ }
      <div className="mt-8">
20        <BarChart data={analytics.statementTypes} />
        <LineChart data={analytics.userRegistrations} />
      </div>
    </div>
  );
25 }

```

Figura 2.14: Componente del dashboard de administración.

Sprint 6 Review

Al finalizar el Sprint 6 y, con ello, el ciclo de desarrollo principal del proyecto, la aplicación cuenta con un dashboard de analíticas completo y funcional. Los administradores ahora pueden obtener una visión clara y cuantitativa del uso del sistema, lo que les permite tomar decisiones informadas sobre su gestión y evolución. La presentación visual de los datos facilita la comprensión de las tendencias y patrones de uso.

Sprint 6 Retrospective

■ **¿Qué salió bien?**

La implementación del dashboard fue un éxito. La separación entre la lógica de negocio para calcular las estadísticas en el backend y la capa de presentación en el frontend funcionó muy bien. El uso de una librería de gráficos facilitó la creación de un dashboard atractivo e informativo con un esfuerzo de desarrollo moderado.

■ **¿Qué se puede mejorar?**

El dashboard actual proporciona una excelente visión general, pero podría ser aún más útil si fuera interactivo. Por ejemplo, permitir a los administradores filtrar las estadísticas por rangos de fechas. Las consultas para calcular las analíticas podrían volverse lentas a medida que el volumen de datos crece; se podría considerar la implementación de un sistema de caché o la pre-cálculo de estadísticas en segundo plano para optimizar el rendimiento.

Capítulo 3

RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1. Resultados

Este capítulo está dedicado a la presentación de resultados y análisis del desarrollo del sistema a través de las funcionalidades implementadas junto a las interfaces de usuario correspondientes en base a las necesidades identificadas dentro de la etapa de requerimientos y la construcción de Historias de Usuario, de tal manera que se garantice la satisfacción del usuario a través de un producto intuitivo y fácil de usar.

En las siguientes secciones se detallarán las pruebas realizadas junto a los resultados obtenidos y la retroalimentación obtenida. Además, se presentarán las conclusiones y recomendaciones producto de la ejecución de este proyecto y las cuales serán claves a futuro para la optimización del mismo o para la realización de proyectos relacionados a la traducción de bases de datos SQL a NoSQL.

3.1.1. Pruebas Funcionales

Dado la extensión del sistema creado, se realizaron 6 casos de prueba enfocados en las funcionalidades principales y que el usuario final utilizará con mayor frecuencia. Estos casos validan el correcto funcionamiento de características como el registro e inicio de sesión, la conexión a la base de datos, la traducción de sentencias SQL a Cypher, la ejecución y visualización de resultados.

En la tabla 3.1 se muestra el caso de prueba dedicado a la traducción de una sentencia SQL que utiliza la operación `INSERT` y la generación de su equivalente en lenguaje Cypher,

lista para ser ejecutada dentro de una base de datos en Neo4j. Aquí se podrán observar el comportamiento del sistema operaciones válidas y no permitidas. El resto de casos de prueba se encuentran en el Anexo IV.

Tabla 3.1: Caso de prueba: Traducir sentencia con operaciones INSERT

Campo	Descripción
ID	CP-03
Historia de Usuario	QTE-04
Nombre	Traducir consultas INSERT
Cumple (Sí/No)	Sí
Descripción de la Prueba	Verificar que el sistema permita traducir consultas INSERT correctamente.
Precondiciones	■ El usuario debe estar autenticado en el sistema. ■ El usuario debe tener una conexión a Neo4j almacenada y activa.
Pasos	■ Acceder a la pantalla de traducción de consultas. ■ Seleccionar una conexión a Neo4j activa. ■ Ingresar una sentencia SQL INSERT válida. ■ Dar clic en el boton Traducir.
Datos de Prueba	Sentencia: INSERT INTO Usuario (nombre, edad) VALUES ('Usuario Nuevo', 20);
Resultados Esperados	El sistema indica el equivalente en lenguaje Cypher junto a la sentencia SQL ingresada.
Resultados Obtenidos	El sistema mostró un mensaje de éxito de consulta traducida y la consulta equivalente en lenguaje Cypher junto a la consulta SQL ingresada, además del tiempo de traducción.

3.1.2. Pruebas de Usabilidad

Por su parte, las pruebas de usabilidad se centran en la evaluación de la experiencia de los usuarios finales al interactuar con las interfaces que conforman el software, la cual permitirá conocer la facilidad de su uso.

Para ello, se contó con la participación de 10 personas que cumplan con los perfiles de usuarios del sistema definidos anteriormente. Utilizando el método *SUS System Usability Scale* se podrá obtener resultados de forma cuantitativa sobre la experiencia del usuario con el *middleware*, mismos que se complementarán con los resultados de pruebas específicas que se enfocan en tareas concretas para medir la facilidad de realizarlas y la eficiencia del sistema al cumplirlas.

Tareas Específicas

Al realizar tareas específicas nos permiten evaluar los siguientes características con respecto al cumplimiento de estas:

- **Tasa de Éxito:** Porcentaje de pasos completados para realizar la tarea asignada.
- **Tiempo Empleado:** Indica el tiempo promedio que los usuarios tardaron una tarea específica.
- **N° Errores:** Número de tareas que no se pudieron completar de forma correcta.
- **Nivel de Satisfacción:** Puntuación asignada por el usuario de forma subjetiva en base a la facilidad y experiencia obtenida al completar la tarea.

La Tabla 3.5 se presenta las tareas indicadas a los usuarios junto a los pasos correspondientes para la evaluación del sistema:

Tabla 3.2: Tareas Específicas para medición de la Usabilidad del *middleware*.

Tarea	Pasos
Tarea 01	■ Navegar a la página de inicio de sesión. ■ Ingresar las credenciales de un usuario registrado. ■ Validar que el inicio de sesión sea exitoso y se redirija a la pantalla principal.

Continúa en la siguiente página...

Tarea	Pasos
Tarea 02	■ Dirigirse al panel de conexiones y seleccionar nueva conexión. ■ Completar los campos requeridos agregar una nueva conexión. ■ Probar conexión y guardarla. ■ Verificar que la conexión haya sido guardada correctamente.
Tarea 03	■ Dirigirse al panel de conexiones y seleccionar una conexión Neo4j existente. ■ Navegar al panel de traducción de consultas. ■ Ingresar una sentencia SQL válida. ■ Seleccionar la opción Traducir. ■ Validar que la consulta SQL haya sido traducida a lenguaje Cypher correctamente y se almacene en el historial.
Tarea 04	■ Dirigirse al panel de traducción de consultas. ■ Ingresar una sentencia SQL válida. ■ Seleccionar la opción Ejecutar. ■ Validar que aparezca la consulta traducida en lenguaje Cypher y los resultados de la ejecución.

3.1.3. System Usability Scalie (SUS)

Para complementar los resultados obtenidos al ejecutar las tareas específicas, se utilizó el sistema SUS, que permite evaluar la usabilidad completa a través de un cuestionario de 10 preguntas, mismas que serán evaluadas utilizando la escala de Likert.

Las preguntas definidas por SUS se pueden observar en la Tabla 3.3:

Tabla 3.3: Preguntas SUS.

Valoración	Descripción
1	Totalmente en desacuerdo
2	En desacuerdo
3	Neutro
4	De acuerdo
5	Totalmente de acuerdo

La escala de Likert que se encuentra en la Tabla 3.4 utilizada tiene un rango de 1 a 5, acorde al grado de satisfacción del usuario:

Tabla 3.4: Escala de Likert.

No.	Pregunta
1	Creo que me gustaría utilizar este sistema con frecuencia
2	Encontré el sistema innecesariamente complejo
3	Pensé que el sistema era fácil de usar
4	Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema
5	Encontré que las diversas funciones de este sistema estaban bien integradas
6	Pensé que había demasiada inconsistencia en este sistema
7	Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente
8	Encontré el sistema muy complicado de usar
9	Me sentí muy seguro usando el sistema
10	Necesitaba aprender muchas cosas antes de empezar con este sistema

Para la obtención de un resultado de acuerdo con SUS, se debe seguir los siguientes pasos:

1. Sumar los puntajes de las preguntas pares e impares.
2. Al resultado de la suma de las preguntas impares se le debe restar 5.
3. El resultado de la suma de las preguntas pares debe ser restado a 25.

4. Se suma los últimos valores obtenidos y se multiplica por 2,5.
5. Si el resultado final es mayor a 67, significa que el sistema es usable.

3.1.4. Resultados de Pruebas Funcionales

A través de los casos de prueba se logró validar que las funcionalidades principales del sistema cumplan con los requisitos definidos dentro de las Historias de Usuario. Cada caso de prueba permitió evaluar los comportamientos que presente el sistema ante los distintos eventos o acciones que pueden ejecutarse por parte del usuario.

Como ejemplo se tomó el caso de prueba CP-03 donde se validó la funcionalidad de probar una consulta SQL con la sentencia `INSERT` válida e inválida para poder verificar que los resultados obtenidos antes ambos escenarios opuestos sean los esperados.

Como se observa en la Figura, para una sentencia válida el sistema devuelve su equivalente en lenguaje Cypher, mientras que para una sentencia inválida se muestra el mensaje de error claro y preciso.

Resultados de Pruebas de Usabilidad

Las pruebas de usabilidad arrojaron resultados positivos en cuanto a la experiencia de los usuarios para interactuar con el sistema. El resumen de resultados de las tareas específicas se indican en la Tabla 3.5 evaluadas en base a las métricas definidas previamente. Para un ver los resultados detallados por participante y por tarea, se puede consultar en el Anexo VI.

Tabla 3.5: Resultados de Tareas Específicas.

Tarea	Tasa de Éxito (%)	Tiempo Promedio (min)	Errores	Satisfacción (1-5)
Tarea 01	100	1.72	3	5
Tarea 02	95	4.43	5	4
Tarea 03	96	3.57	3	5
Tarea 04	98	2.86	3	4

A través de estos resultados se pudo determinar una tasa de éxito del 97 %, un tiempo de promedio de 3,14 minutos por tarea, un promedio de 0,43 errores por tarea y una satisfacción de 4,4 sobre 5. Esto se traduce en que los usuarios lograron completar sin mayor dificultad las tareas asignadas y de forma rápida, generando una experiencia positiva al

interactuar con el sistema. A pesar de ello, la tarea 2 presentó la mayor cantidad de errores y el tiempo promedio más alto para completarla, lo cual nos indica un área de mejora o tal vez un ajuste en las indicaciones de los pasos.

Resultados de Cuestionario SUS

Utilizando los datos del participante 3 para el ejemplo, se indicará el cálculo del puntaje SUS:

$$P_i = 4 + 4 + 4 + 4 + 4 = 20 \quad (3.1)$$

$$P_p = 3 + 2 + 2 + 2 + 2 = 11 \quad (3.2)$$

$$SP_i = 20 - 5 = 15 \quad (3.3)$$

$$SP_p = 25 - 11 = 14 \quad (3.4)$$

$$Promedio_{SUS} = (15 + 14) \times 2,5 = 72,5 \quad (3.5)$$

En la Tabla 3.6 se presenta la puntuación asignada por los encuestados a cada una de las preguntas y el promedio SUS obtenido por cada participante.

Tabla 3.6: Resultados de la encuesta SUS.

Participante	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	SUS
Participante 1	4	2	4	2	4	2	5	1	4	2	80
Participante 2	5	2	4	1	5	2	4	2	5	1	87.5
Participante 3	4	3	4	2	4	2	4	2	4	2	72.5
Participante 4	4	2	5	1	4	1	5	3	5	2	85
Participante 5	4	2	4	2	4	2	4	2	4	2	75
Participante 6	4	3	3	2	4	3	4	2	4	2	67.5
Participante 7	5	2	5	2	5	1	5	2	5	2	90
Participante 8	4	2	4	2	4	2	5	2	4	2	77.5
Participante 9	4	2	4	3	4	2	4	2	4	2	72.5
Participante 10	4	2	5	1	5	2	4	2	5	1	87.5

El promedio final de la aplicación fue de **79.5**, lo cual refleja que los usuarios consideran que es un sistema fácil de usar y eficiente para cumplir con las necesidades para la cuales fue desarrollado.

3.2. Conclusiones

El presente Trabajo de Integración Curricular (TIC) permitió llevar a cabo un trabajo que involucra cada una de las etapas de desarrollo de software desde la investigación inicial hasta la implementación del *middleware* y validación del sistema construido.

Durante la investigación para la traducción de bases de datos relacionales a bases de datos orientadas a grafos, se pudo conocer más a detalle las características y ventajas que estas últimas presentan dado la velocidad y precisión que brinda en la consulta de datos que presenten relaciones complejas.

Así mismo, la elección de una metodología ágil en conjunto de un marco de trabajo como Scrum, permitió una construcción incremental del sistema y de manera ágil, que permitió adaptar sin mayor dificultad los cambios que iban surgiendo a lo largo del proyecto.

La separación clara entre el *frontend* y el *backend* durante el diseño de la arquitectura del sistema, permite que el código pueda ser altamente escalable y mantenible, lo cual es para la integración con bibliotecas externas y otras bases de datos en versiones futuras.

Durante la etapa de desarrollo, la implementación de librerías dedicadas a la conexión a bases de datos así como al parseo de de sintaxis SQL facilitó que la aplicación soporte una gramática extensa del lenguaje SQL incluyendo todas las operaciones DML, cláusulas como JOIN, ORDER BY, HAVING y funciones de agregación como COUNT o SUM. Esto añadido a la fácil integración con DBMS como SQL Server y Neo4j, resultaron en el desarrollo de un motor de traducción bastante robusto y capaz de ejecutar sentencias Cypher traducidas en una base de datos Neo4j sin mayor complejidad.

Tanto los resultados de las pruebas funcionales como las pruebas de usabilidad demostraron la integración de los componentes del sistema para producto que satisfaga las necesidades de los usuarios de una manera atractiva y familiar. Así también, con la retroalimentación adquirida, se pueden identificar áreas de mejora que permitirán optimizar el sistema en versiones futuras.

El proyecto no solo cumplió con los objetivos planteados inicialmente sino que también aportó un aprendizaje valioso en cuanto a la traducción de bases de datos orientadas a grafos, la gestión y construcción de un producto software. La integración entre metodologías de

trabajo, herramientas y conocimientos técnicos garantizan un resultado exitoso y adaptable ante las demandas cambiantes dentro del software moderno.

3.3. Recomendaciones

Una vez finalizado el desarrollo del *middleware* y en base al trabajo realizado, se plantean las siguientes recomendaciones que permitirán fortalecer y ampliar las características del producto software realizado.

- Para mejorar el método de trabajo se recomienda la inversión en la capacitación de los miembros del equipo sobre tecnologías emergentes y otros conocimientos que aumenten su productividad y rendimiento, además de fomentar un ambiente comunicativo que permita tener una visión más amplia y clara del desafío de negocio, mismo que ayudará a realizar un análisis riguroso y detallado durante la toma de decisiones al construir un producto.
- Para el uso de conexiones muy frecuentes a bases de datos, se recomienda la implementación para seleccionar una base de datos como predeterminada, con esto se evita la búsqueda y selección de forma manual en reiteradas ocasiones, aumentando la familiaridad e interés de uso de los usuarios.
- También se recomienda la extensión de la gramática SQL soportada para la traducción, incluyendo operaciones DDL útil para el manejo completo de tablas y esquemas dentro de las bases de datos SQL, mismos que son útiles para un control total de los datos dentro de una base de datos.
- Debido a su fácil integración tanto con bases de datos relacionales como orientadas a grafos, se recomienda la inclusión de otros dialectos y la compatibilidad con otros DBMS como Postgres, MySQL, Amazon Neptune o ArangoDB.
- Se recomienda la implementación de pruebas de estrés que permitan identificar posibles cuellos de botella durante el proceso de traducción y ejecución de consultas, además del renderizado de resultados a grafos.

Capítulo 4

REFERENCIAS BIBLIOGRÁFICAS

- [1] J. Dai, «SQL to NoSQL: What to do and How,» en *IOP Conference Series: Earth and Environmental Science*, IOP Publishing, vol. 234, 2019, pág. 012 080.
- [2] B. Namdeo y U. Suman, «A Middleware Model for SQL to NoSQL Query Translation,» *Indian Journal of Science and Technology*, vol. 15, n.º 16, págs. 718-728, 2022.
- [3] M. Đukić, O. Pantelić, A. Pajić Simović, S. Krstović y O. Jejić, «A systematic approach for converting relational to graph databases,» 2024.
- [4] R. C. Martin, *Clean architecture: a craftsman's guide to software structure and design*. Prentice Hall Press, 2017.
- [5] R. S. Pressman, *Software engineering: a practitioner's approach*. Palgrave macmillan, 2005.
- [6] A. Stellman y J. Greene, *Learning agile: Understanding scrum, XP, lean, and kanban*. .O'Reilly Media, Inc.", 2014.
- [7] K. Schwaber y J. Sutherland, «The scrum guide,» *Scrum Alliance*, vol. 21, n.º 1, págs. 1-38, 2011.
- [8] P. Ackermann, *Full Stack Web Development: The Comprehensive Guide*. Rheinwerk Publishing Incorporated, 2023.
- [9] J. Goldberg, *Learning TypeScript*. .O'Reilly Media, Inc.", 2022.
- [10] A. Banks y E. Porcello, *Learning React: modern patterns for developing React apps*. O'Reilly Media, 2020.
- [11] Vercel, *Next.js docs*. dirección: <https://nextjs.org/docs>

- [12] K. Bhat, *Ultimate Tailwind CSS Handbook: Build sleek and modern websites with immersive UIs using Tailwind CSS*. Orange Education Pvt Limited, 2023.
- [13] M. Lutz, *Learning python: Powerful object-oriented programming*. "O'Reilly Media, Inc.", 2013.
- [14] B. Lubanovic, *FastAPI*. "O'Reilly Media, Inc.", 2023.
- [15] H. Garcia-Molina, *Database systems: the complete book*. Pearson Education India, 2008.
- [16] A. Silberschatz, H. F. Korth y S. Sudarshan, *Database system concepts*. McGraw-Hill Education, 2011.
- [17] C. M. Ricardo, *Bases de datos*. McGraw Hill Educación, 2009.
- [18] D. Sullivan, *NoSQL for mere mortals*. Addison-Wesley Professional, 2015.
- [19] T. Hills, *NoSQL and SQL data modeling: bringing together data, semantics, and software*. Technics Publications, LLC, 2016.
- [20] P. J. Sadalage y M. Fowler, *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.
- [21] M. Lal, *Neo4j graph data modeling*. Packt Publishing Ltd, 2015.
- [22] A. Vukotic, N. Watt, T. Abedrabbo, D. Fox y J. Partner, *Neo4j in action*. Manning Publications Co., 2014.
- [23] R. Van Bruggen, *Learning Neo4j*. Packt Publishing Ltd, 2014.
- [24] J. Baton y R. Van Bruggen, *Learning Neo4j 3. x: Effective data modeling, performance tuning and data visualization techniques in Neo4j*. Packt Publishing Ltd, 2017.
- [25] F. Staiano, *Designing and Prototyping Interfaces with Figma: Learn essential UX/UI design principles by creating interactive prototypes for mobile, tablet, and desktop*. Packt Publishing Ltd, 2022.
- [26] S. Chacon y B. Straub, *Pro git*. Springer Nature, 2014.
- [27] M. Kennedy y M. Harrison, *Effective PyCharm: Learn the PyCharm IDE with a Hands-on Approach*. Effectivepycharm. com., 2019.
- [28] D. Westerveld, *API Testing and Development with Postman*. Packt Publishing, 2021.
- [29] I. Sommerville, «Software engineering 9th Edition,» ISBN-10, vol. 137035152, pág. 18, 2011.

- [30] M. Jain, A. Khanchandani y C. Rodrigues, «Performance Comparison of Graph Database and Relational Database,» *Computer Science Department San Jose State University*, 2023.
- [31] M. Singh y K. Kaur, «SQL2Neo: Moving health-care data from relational to graph databases,» en *2015 IEEE International Advance Computing Conference (IACC)*, IEEE, 2015, págs. 721-725.

ANEXOS

ANEXO I: Historias de Usuario

ANEXO II: Prototipo de interfaces de usuario en Figma

ANEXO III: Repositorio del Middleware

ANEXO IV: Casos de prueba

ANEXO V: Resultados de pruebas de funcionalidad

ANEXO VI: Resultados de pruebas de usabilidad